

CONTEXT: PROJECT STRUCTURE

Below is the file tree. Use this to understand module relationships.

```
.
├── launch.bat
├── scratch
│   └── generate_icons.py
├── src
│   ├── App.tsx
│   ├── components
│   │   ├── CrawlerPanel.tsx
│   │   ├── JournalSourcesView.tsx
│   │   ├── LetterEditor.tsx
│   │   ├── PenpalPanel.tsx
│   │   ├── ScaffoldModal.tsx
│   │   ├── WorkspacePanel.tsx
│   │   └── blocks
│   │       └── schema.tsx
│   ├── main.tsx
│   ├── services
│   │   ├── journalService.ts
│   │   ├── letterService.ts
│   │   ├── penpalService.ts
│   │   └── workspaceService.ts
│   ├── stores
│   │   └── appStore.ts
│   ├── styles
│   │   ├── global.css
│   │   └── tokens.css
│   ├── types
│   │   └── index.ts
│   ├── utils
│   │   └── serialization.ts
│   └── vite-env.d.ts
├── src-tauri
│   ├── Cargo.toml
│   ├── build.rs
│   └── src
│       ├── commands
│       │   ├── journal.rs
│       │   ├── letter.rs
│       │   ├── mod.rs
│       │   ├── penpal.rs
│       │   └── workspace.rs
│       ├── db.rs
│       ├── lib.rs
│       ├── main.rs
│       ├── migrations
│       │   └── 001_initial.sql
│       ├── parser
│       │   ├── format1.rs
│       │   └── format2.rs
```

```
|
|
|└─ format3.rs
|   └─ mod.rs
└─ state.rs
└─ vibe_snapshot_env.txt
└─ vite.config.ts
```

```
--- START_FILE: launch.bat ---
```

```
@echo off
title Journal-to-Penpal Pipeline Launcher
echo =====
echo   Starting Journal-to-Penpal Pipeline Dev Server...
echo =====
echo.
npm run tauri dev
if %errorlevel% neq 0 (
    echo.
    echo Launch failed with error code %errorlevel%.
    pause
)
```

```
--- END_FILE ---
```

```
--- START_FILE: scratch/generate_icons.py ---
```

```
import os
from PIL import Image

source_path =
r"C:\Users\USER\.gemini\antigravity\brain\0bcfeccf-94e7-42fd-90ea-aa382eb99642\app_icon_
1779453671602.png"
output_dir = r"c:\Users\USER\Desktop\APPS\journal-to-penpal-pipeline\src-tauri\icons"

if not os.path.exists(output_dir):
    os.makedirs(output_dir)

print(f"Loading source image from: {source_path}")
img = Image.open(source_path)

# Ensure it's RGBA
if img.mode != 'RGBA':
    img = img.convert('RGBA')

# 1. 32x32.png
print("Generating 32x32.png...")
img_32 = img.resize((32, 32), Image.Resampling.LANCZOS)
img_32.save(os.path.join(output_dir, "32x32.png"), "PNG")

# 2. 128x128.png
print("Generating 128x128.png...")
img_128 = img.resize((128, 128), Image.Resampling.LANCZOS)
img_128.save(os.path.join(output_dir, "128x128.png"), "PNG")

# 3. 128x128@2x.png (256x256)
print("Generating 128x128@2x.png...")
img_256 = img.resize((256, 256), Image.Resampling.LANCZOS)
```

```

img_256.save(os.path.join(output_dir, "128x128@2x.png"), "PNG")

# 4. icon.ico
print("Generating icon.ico...")
# ICO standard sizes: 16, 32, 48, 64, 128, 256
ico_sizes = [16, 32, 48, 64, 128, 256]
ico_images = []
for size in ico_sizes:
    ico_images.append(img.resize((size, size), Image.Resampling.LANCZOS))
ico_images[0].save(
    os.path.join(output_dir, "icon.ico"),
    format="ICO",
    sizes=[(size, size) for size in ico_sizes],
    append_images=ico_images[1:]
)

# 5. icon.icns
print("Generating icon.icns...")
# ICNS can be saved directly by Pillow
try:
    icns_sizes = [16, 32, 64, 128, 256, 512]
    icns_images = []
    for size in icns_sizes:
        icns_images.append(img.resize((size, size), Image.Resampling.LANCZOS))
    # Filter out only the ones that icns format accepts
    img_512 = img.resize((512, 512), Image.Resampling.LANCZOS)
    img_512.save(os.path.join(output_dir, "icon.icns"), format="ICNS")
except Exception as e:
    print(f"Could not generate true ICNS: {e}. Writing a placeholder fallback file.")
    # Fallback to copy the 512 png or write a simple file
    img_512 = img.resize((512, 512), Image.Resampling.LANCZOS)
    img_512.save(os.path.join(output_dir, "icon.icns"), "PNG")

print("All icons successfully generated in src-tauri/icons!")

--- END_FILE ---

--- START_FILE: src-tauri/Cargo.toml ---

[package]
name = "journal-to-penpal-pipeline"
version = "0.1.0"
description = "A Tauri v2 desktop app for transforming journal entries into penpal letters"
authors = ["you"]
edition = "2021"

[lib]
name = "journal_to_penpal_pipeline_lib"
crate-type = ["lib", "cdylib", "staticlib"]

```

```
[build-dependencies]
tauri-build = { version = "2", features = [] }

[dependencies]
tauri = { version = "2", features = [] }
tauri-plugin-dialog = "2"
tauri-plugin-fs = "2"
tauri-plugin-clipboard-manager = "2"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
rusqlite = { version = "0.33", features = ["bundled"] }
chrono = { version = "0.4", features = ["serde"] }
uuid = { version = "1", features = ["v4"] }
nom = "7.1"
nom_locate = "4"
```

```
--- END_FILE ---
```

```
--- START_FILE: src-tauri/build.rs ---
```

```
fn main() {
    tauri_build::build()
}
```

```
--- END_FILE ---
```

```
--- START_FILE: src-tauri/src/commands/journal.rs ---
```

```
use crate::state::AppState;
use crate::parser::{self, ThoughtUnit};
use serde::{Deserialize, Serialize};
use std::fs;
use std::path::Path;
use chrono::{DateTime, Utc};
use uuid::Uuid;

#[derive(Serialize, Deserialize, Debug, Clone)]
#[serde(rename_all = "camelCase")]
pub struct ImportResult {
    pub entry_count: i32,
    pub format_version: i32,
    pub warnings: Vec<String>,
}

#[derive(Serialize, Deserialize, Debug, Clone)]
#[serde(rename_all = "camelCase")]
pub struct JournalSource {
    pub id: String,
    pub file_path: String,
```

```

    pub file_name: String,
    pub last_modified: String,
    pub last_imported: String,
    pub format_version: i32,
    pub entry_count: i32,
}

#[tauri::command]
pub fn import_journal_file(
    state: tauri::State<AppState>,
    file_path: String,
) -> Result<ImportResult, String> {
    // 1. File Type Guard (from PRD & Atlas)
    let path = Path::new(&file_path);
    let ext = path.extension().and_then(|e| e.to_str()).unwrap_or("");
    if ext != ".md" && ext != ".txt" {
        return Err("Invalid file type. Only .md and .txt files are
allowed.".to_string());
    }

    // 2. Read file content
    let content = fs::read_to_string(&file_path)
        .map_err(|e| format!("Could not read file: {}", e))?;

    // 3. Get metadata and last modified time
    let metadata = fs::metadata(&file_path)
        .map_err(|e| format!("Could not read file metadata: {}", e))?;
    let last_modified_dt: DateTime<Utc> = metadata.modified()
        .map_err(|e| format!("Could not read modification time: {}", e))?
        .into();
    let last_modified = last_modified_dt.to_rfc3339();
    let last_imported = Utc::now().to_rfc3339();

    // 4. Parse journal
    let (thought_units, format_version, warnings) = parser::parse_journal(&content,
&file_path);

    // 5. Database transaction (atomic re-import: delete + re-insert)
    let mut conn = state.db.lock().map_err(|e| e.to_string())?;
    let tx = conn.transaction().map_err(|e| e.to_string())?;

    // Delete existing
    tx.execute("DELETE FROM thought_units WHERE source_file_path = ?", [&file_path])
        .map_err(|e| format!("DB Error (deleting old entries): {}", e))?;
    tx.execute("DELETE FROM journal_sources WHERE file_path = ?", [&file_path])
        .map_err(|e| format!("DB Error (deleting old source): {}", e))?;

    // Insert source metadata
    let source_id = Uuid::new_v4().to_string();
    let file_name = path.file_name()
        .and_then(|n| n.to_str())
        .unwrap_or("Unknown")
        .to_string();

```

```

    tx.execute(
        "INSERT INTO journal_sources (id, file_path, file_name, last_modified,
last_imported, format_version, entry_count)
        VALUES (?, ?, ?, ?, ?, ?, ?)",
        rusqlite::params![
            source_id,
            file_path,
            file_name,
            last_modified,
            last_imported,
            format_version as i32,
            thought_units.len() as i32,
        ],
    ).map_err(|e| format!("DB Error (inserting source metadata): {}", e))?;

// Insert thought units
for tu in &thought_units {
    tx.execute(
        "INSERT INTO thought_units (id, date, content, category, source_file_path,
source_line_number, format_version, created_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
        rusqlite::params![
            tu.id,
            tu.date,
            tu.content,
            tu.category,
            tu.source_file_path,
            tu.source_line_number,
            tu.format_version,
            tu.created_at,
        ],
    ).map_err(|e| format!("DB Error (inserting entry): {}", e))?;
}

tx.commit().map_err(|e| format!("DB Transaction Commit failed: {}", e))?;

Ok(ImportResult {
    entry_count: thought_units.len() as i32,
    format_version: format_version as i32,
    warnings,
})
}

#[tauri::command]
pub fn get_import_status(state: tauri::State<AppState>) -> Result<Vec<JournalSource>,
String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;
    let mut stmt = conn.prepare(
        "SELECT id, file_path, file_name, last_modified, last_imported, format_version,
entry_count
        FROM journal_sources ORDER BY last_imported DESC"
    ).map_err(|e| e.to_string())?;

```

```

let rows = stmt.query_map([], |row| {
    Ok(JournalSource {
        id: row.get(0)?,
        file_path: row.get(1)?,
        file_name: row.get(2)?,
        last_modified: row.get(3)?,
        last_imported: row.get(4)?,
        format_version: row.get(5)?,
        entry_count: row.get(6)?,
    })
}).map_err(|e| e.to_string())?;

let mut sources = Vec::new();
for r in rows {
    sources.push(r.map_err(|e| e.to_string())?);
}
Ok(sources)
}

#[tauri::command]
pub fn remove_journal_source(
    state: tauri::State<AppState>,
    file_path: String,
) -> Result<bool, String> {
    let mut conn = state.db.lock().map_err(|e| e.to_string())?;
    let tx = conn.transaction().map_err(|e| e.to_string())?;

    tx.execute("DELETE FROM thought_units WHERE source_file_path = ?", [&file_path])
        .map_err(|e| e.to_string())?;
    tx.execute("DELETE FROM journal_sources WHERE file_path = ?", [&file_path])
        .map_err(|e| e.to_string())?;

    tx.commit().map_err(|e| e.to_string())?;
    Ok(true)
}

#[tauri::command]
pub fn get_thought_units(
    state: tauri::State<AppState>,
    start_date: Option<String>,
    end_date: Option<String>,
    category: Option<String>,
) -> Result<Vec<ThoughtUnit>, String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;

    let mut query = "SELECT id, date, content, category, source_file_path,
source_line_number, format_version, created_at
    FROM thought_units WHERE 1=1".to_string();

    let mut params = Vec::new();
    if let Some(ref start) = start_date {
        query.push_str(" AND date >= ?");
    }

```



```

        params.push(start);
    }
    if let Some(ref end) = end_date {
        query.push_str(" AND date <= ?");
        params.push(end);
    }
    if let Some(ref cat) = category {
        query.push_str(" AND category = ?");
        params.push(cat);
    }

    query.push_str(" ORDER BY date ASC, source_line_number ASC");

    let mut stmt = conn.prepare(&query).map_err(|e| e.to_string())?;
    let rows = stmt.query_map(rusqlite::params_from_iter(params), |row| {
        Ok(ThoughtUnit {
            id: row.get(0)?,
            date: row.get:::<_, Option<String>>(1)?,
            content: row.get(2)?,
            category: row.get(3)?,
            source_file_path: row.get(4)?,
            source_line_number: row.get(5)?,
            format_version: row.get(6)?,
            created_at: row.get(7)?,
        })
    }).map_err(|e| e.to_string())?;

    let mut thought_units = Vec::new();
    for r in rows {
        thought_units.push(r.map_err(|e| e.to_string())?);
    }
    Ok(thought_units)
}

```

--- END_FILE ---

--- START_FILE: src-tauri/src/commands/letter.rs ---

```

use crate::state::AppState;
use serde::{Deserialize, Serialize};
use uuid::Uuid;
use chrono::Utc;

#[derive(Serialize, Deserialize, Debug, Clone)]
#[serde(rename_all = "camelCase")]
pub struct Letter {
    pub id: String,
    pub penpal_id: String,
    pub status: String,
    pub created_at: String,
    pub updated_at: String,
}

```

```

}

#[derive(Serialize, Deserialize, Debug, Clone)]
#[serde(rename_all = "camelCase")]
pub struct LetterContentResponse {
    pub blocks_json: String,
}

#[tauri::command]
pub fn create_letter(
    state: tauri::State<AppState>,
    penpal_id: String,
) -> Result<Letter, String> {
    let mut conn = state.db.lock().map_err(|e| e.to_string())?;
    let tx = conn.transaction().map_err(|e| e.to_string())?;

    // Check if a draft letter for this penpal already exists
    let existing_letter: Result<(String, String, String), rusqlite::Error> =
tx.query_row(
    "SELECT id, created_at, updated_at FROM letters WHERE penpal_id = ? AND status = 'draft'",
    [&penpal_id],
    |row| Ok((row.get(0)?, row.get(1)?, row.get(2)?)),
);

    if let Ok((letter_id, created_at, updated_at)) = existing_letter {
        tx.rollback().map_err(|e| e.to_string())?;
        return Ok(Letter {
            id: letter_id,
            penpal_id,
            status: "draft".to_string(),
            created_at,
            updated_at,
        });
    }

    let letter_id = Uuid::new_v4().to_string();
    let content_id = Uuid::new_v4().to_string();
    let now = Utc::now().to_rfc3339();

    // Create the letter
    tx.execute(
        "INSERT INTO letters (id, penpal_id, status, created_at, updated_at)
        VALUES (?, ?, 'draft', ?, ?)",
        rusqlite::params![letter_id, penpal_id, now, now],
    ).map_err(|e| format!("Failed to create letter in DB: {}", e))?;

    // Create default letter content
    tx.execute(
        "INSERT INTO letter_content (id, letter_id, blocks_json)
        VALUES (?, ?, '[]')",
        rusqlite::params![content_id, letter_id],
    ).map_err(|e| format!("Failed to create letter content: {}", e))?;

```

```

tx.commit().map_err(|e| e.to_string())?;

Ok(Letter {
    id: letter_id,
    penpal_id,
    status: "draft".to_string(),
    created_at: now.clone(),
    updated_at: now,
})
}

#[tauri::command]
pub fn save_letter_content(
    state: tauri::State<AppState>,
    letter_id: String,
    blocks_json: String,
) -> Result<bool, String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;

    // Atomic insert/replace for letter_content
    // We use INSERT OR REPLACE as defined in BACKEND_STRUCTURE.md § Storage Rules
    conn.execute(
        "INSERT OR REPLACE INTO letter_content (id, letter_id, blocks_json)
        VALUES (
            COALESCE((SELECT id FROM letter_content WHERE letter_id = ?), ?),
            ?,
            ?
        )",
        rusqlite::params![letter_id, Uuid::new_v4().to_string(), letter_id,
blocks_json],
    ).map_err(|e| format!("Failed to save letter content: {}", e))?;

    // Update the letter's updated_at timestamp
    conn.execute(
        "UPDATE letters SET updated_at = ? WHERE id = ?",
        rusqlite::params![Utc::now().to_rfc3339(), letter_id],
    ).map_err(|e| format!("Failed to update letter metadata: {}", e))?;

    Ok(true)
}

#[tauri::command]
pub fn load_letter_content(
    state: tauri::State<AppState>,
    letter_id: String,
) -> Result<LetterContentResponse, String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;
    let blocks_json: String = conn.query_row(
        "SELECT blocks_json FROM letter_content WHERE letter_id = ?",
        [&letter_id],
        |row| row.get(0),
    ).map_err(|e| format!("Letter content not found: {}", e))?;

```

```

    Ok(LetterContentResponse { blocks_json })
}

#[tauri::command]
pub fn mark_letter_sent(
    state: tauri::State<AppState>,
    letter_id: String,
) -> Result<bool, String> {
    let mut conn = state.db.lock().map_err(|e| e.to_string())?;
    let tx = conn.transaction().map_err(|e| e.to_string())?;

    // Update letter status to sent
    tx.execute(
        "UPDATE letters SET status = 'sent', updated_at = ? WHERE id = ?",
        rusqlite::params![Utc::now().to_rfc3339(), letter_id],
    ).map_err(|e| format!("Failed to mark letter as sent: {}", e))?;

    // Delete content from letter_content (or keep it? In BACKEND_STRUCTURE.md, ON
    DELETE CASCADE works on letter delete,
    // but the letter status becomes "sent", meaning the letter still exists in letters
    table, just status is "sent").
    // Closed tabs/sent letters are read-only, so we keep content.

    tx.commit().map_err(|e| e.to_string())?;
    Ok(true)
}

--- END_FILE ---

--- START_FILE: src-tauri/src/commands/mod.rs ---

pub mod journal;
pub mod penpal;
pub mod letter;
pub mod workspace;

--- END_FILE ---

--- START_FILE: src-tauri/src/commands/penpal.rs ---

use crate::state::AppState;
use serde::{Deserialize, Serialize};
use uuid::Uuid;
use chrono::Utc;

#[derive(Serialize, Deserialize, Debug, Clone)]
#[serde(rename_all = "camelCase")]
pub struct Penpal {

```

```

pub id: String,
pub name: String,
pub country: String,
pub interests: String,
pub topics: String,
pub notes: String,
pub created_at: String,
pub updated_at: String,
pub letter_count: Option<i64>,
pub last_letter_date: Option<String>,
}

#[derive(Serialize, Deserialize, Debug, Clone)]
#[serde(rename_all = "camelCase")]
pub struct Correspondence {
    pub id: String,
    pub penpal_id: String,
    pub direction: String, // "sent" | "received"
    pub content: String,
    pub letter_date: String,
    pub imported_at: String,
}

#[tauri::command]
pub fn create_penpal(
    state: tauri::State<AppState>,
    name: String,
    country: Option<String>,
    interests: Option<String>,
    topics: Option<String>,
    notes: Option<String>,
) -> Result<Penpal, String> {
    if name.trim().is_empty() {
        return Err("Penpal name is required.".to_string());
    }

    let id = Uuid::new_v4().to_string();
    let country = country.unwrap_or_default();
    let interests = interests.unwrap_or_default();
    let topics = topics.unwrap_or_default();
    let notes = notes.unwrap_or_default();
    let now = Utc::now().to_rfc3339();

    let conn = state.db.lock().map_err(|e| e.to_string())?;
    conn.execute(
        "INSERT INTO penpals (id, name, country, interests, topics, notes, created_at,
updated_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
        rusqlite::params![id, name, country, interests, topics, notes, now, now],
    ).map_err(|e| format!("Failed to create penpal in DB: {}", e))?;

    Ok(Penpal {
        id,

```

```

        name,
        country,
        interests,
        topics,
        notes,
        created_at: now.clone(),
        updated_at: now,
        letter_count: Some(0),
        last_letter_date: None,
    })
}

#[tauri::command]
pub fn get_penpals(state: tauri::State<AppState>) -> Result<Vec<Penpal>, String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;
    let mut stmt = conn.prepare(
        "SELECT p.id, p.name, p.country, p.interests, p.topics, p.notes, p.created_at,
        p.updated_at,
            (SELECT COUNT(*) FROM correspondence WHERE penpal_id = p.id) as
        letter_count,
            (SELECT MAX(letter_date) FROM correspondence WHERE penpal_id = p.id) as
        last_letter_date
        FROM penpals p ORDER BY p.name ASC"
    ).map_err(|e| e.to_string())?;

    let rows = stmt.query_map([], |row| {
        Ok(Penpal {
            id: row.get(0)?,
            name: row.get(1)?,
            country: row.get(2)?,
            interests: row.get(3)?,
            topics: row.get(4)?,
            notes: row.get(5)?,
            created_at: row.get(6)?,
            updated_at: row.get(7)?,
            letter_count: Some(row.get(8)?),
            last_letter_date: row.get(9)?,
        })
    }).map_err(|e| e.to_string())?;

    let mut penpals = Vec::new();
    for r in rows {
        penpals.push(r.map_err(|e| e.to_string())?);
    }
    Ok(penpals)
}

#[tauri::command]
pub fn update_penpal(
    state: tauri::State<AppState>,
    id: String,
    name: String,
    country: Option<String>,

```

```

    interests: Option<String>,
    topics: Option<String>,
    notes: Option<String>,
) -> Result<Penpal, String> {
    if name.trim().is_empty() {
        return Err("Penpal name is required.".to_string());
    }

    let country = country.unwrap_or_default();
    let interests = interests.unwrap_or_default();
    let topics = topics.unwrap_or_default();
    let notes = notes.unwrap_or_default();
    let now = Utc::now().to_rfc3339();

    let conn = state.db.lock().map_err(|e| e.to_string())?;

    // Check if penpal exists and get its created_at time
    let created_at: String = conn.query_row(
        "SELECT created_at FROM penpals WHERE id = ?",
        [&id],
        |row| row.get(0)
    ).map_err(|_| "Penpal not found.".to_string())?;

    conn.execute(
        "UPDATE penpals
        SET name = ?, country = ?, interests = ?, topics = ?, notes = ?, updated_at = ?
        WHERE id = ?",
        rusqlite::params![name, country, interests, topics, notes, now, id],
    ).map_err(|e| format!("Failed to update penpal in DB: {}", e))?;

    // Query metrics to avoid losing them
    let (letter_count, last_letter_date): (i64, Option<String>) = conn.query_row(
        "SELECT
        (SELECT COUNT(*) FROM correspondence WHERE penpal_id = ?),
        (SELECT MAX(letter_date) FROM correspondence WHERE penpal_id = ?)",
        rusqlite::params![id, id],
        |row| Ok((row.get(0)?, row.get(1)?)),
    ).unwrap_or((0, None));

    Ok(Penpal {
        id,
        name,
        country,
        interests,
        topics,
        notes,
        created_at,
        updated_at: now,
        letter_count: Some(letter_count),
        last_letter_date,
    })
}

```

```

#[tauri::command]
pub fn add_correspondence(
    state: tauri::State<AppState>,
    penpal_id: String,
    direction: String,
    content: String,
    letter_date: String,
) -> Result<Correspondence, String> {
    if content.trim().is_empty() {
        return Err("Correspondence content cannot be empty.".to_string());
    }
    if direction != "sent" && direction != "received" {
        return Err("Direction must be either 'sent' or 'received'.".to_string());
    }

    let id = Uuid::new_v4().to_string();
    let imported_at = Utc::now().to_rfc3339();

    let conn = state.db.lock().map_err(|e| e.to_string())?;
    conn.execute(
        "INSERT INTO correspondence (id, penpal_id, direction, content, letter_date, imported_at)
        VALUES (?, ?, ?, ?, ?, ?)",
        rusqlite::params![id, penpal_id, direction, content, letter_date, imported_at],
    ).map_err(|e| format!("Failed to add correspondence: {}", e))?;

    Ok(Correspondence {
        id,
        penpal_id,
        direction,
        content,
        letter_date,
        imported_at,
    })
}

#[tauri::command]
pub fn get_correspondence(
    state: tauri::State<AppState>,
    penpal_id: String,
) -> Result<Vec<Correspondence>, String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;
    let mut stmt = conn.prepare(
        "SELECT id, penpal_id, direction, content, letter_date, imported_at
        FROM correspondence
        WHERE penpal_id = ?
        ORDER BY letter_date ASC"
    ).map_err(|e| e.to_string())?;

    let rows = stmt.query_map([&penpal_id], |row| {
        Ok(Correspondence {
            id: row.get(0)?,
            penpal_id: row.get(1)?,

```



```

        direction: row.get(2)?,
        content: row.get(3)?,
        letter_date: row.get(4)?,
        imported_at: row.get(5)?,
    })
}).map_err(|e| e.to_string())?;

let mut list = Vec::new();
for r in rows {
    list.push(r.map_err(|e| e.to_string())?);
}
Ok(list)
}

```

--- END_FILE ---

--- START_FILE: src-tauri/src/commands/workspace.rs ---

```

use crate::state::AppState;
use serde::{Deserialize, Serialize};
use chrono::Utc;

#[derive(Serialize, Deserialize, Debug, Clone)]
#[serde(rename_all = "camelCase")]
pub struct WorkspaceState {
    pub open_letter_ids: String, // Serialized JSON array of letter IDs
    pub active_letter_id: Option<String>,
    pub crawler_state: String,   // Serialized JSON of crawler state
}

#[tauri::command]
pub fn save_workspace_state(
    state: tauri::State<AppState>,
    open_letter_ids: String,
    active_letter_id: Option<String>,
    crawler_state: String,
) -> Result<bool, String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;

    // We use INSERT OR REPLACE for the singleton row (id = 1)
    conn.execute(
        "INSERT OR REPLACE INTO workspace_state (id, open_letter_ids, active_letter_id, crawler_state, updated_at)
        VALUES (1, ?, ?, ?, ?)",
        rusqlite::params![
            open_letter_ids,
            active_letter_id,
            crawler_state,
            Utc::now().to_rfc3339()
        ],
    ).map_err(|e| format!("Failed to save workspace state: {}", e))?;
}

```

```

    Ok(true)
}

#[tauri::command]
pub fn load_workspace_state(state: tauri::State<AppState>) -> Result<WorkspaceState, String> {
    let mut conn = state.db.lock().map_err(|e| e.to_string())?;

    // Query existing singleton or create it if missing
    let raw_state: Result<(String, Option<String>, String), rusqlite::Error> =
conn.query_row(
    "SELECT open_letter_ids, active_letter_id, crawler_state FROM workspace_state
WHERE id = 1",
    [],
    |row| Ok((row.get(0)?, row.get(1)?, row.get(2)?)),
);

    let (open_ids_str, active_letter_id, crawler_state) = match raw_state {
        Ok(data) => data,
        Err(_) => {
            // Default empty state
            ("[]".to_string(), None, "{}".to_string())
        }
    };

    // Edge Cases: Clean orphaned references (silently filter against existing letters
table)
    let open_ids: Vec<String> = serde_json::from_str(&open_ids_str).unwrap_or_default();
    let mut valid_open_ids = Vec::new();

    for id in open_ids {
        let exists: bool = conn.query_row(
            "SELECT EXISTS(SELECT 1 FROM letters WHERE id = ?)",
            [&id],
            |row| row.get(0),
        ).unwrap_or(false);

        if exists {
            valid_open_ids.push(id);
        }
    }

    let final_active = if let Some(ref active_id) = active_letter_id {
        if valid_open_ids.contains(active_id) {
            Some(active_id.clone())
        } else {
            None
        }
    } else {
        None
    };
};

```

```

    let final_open_ids_str = serde_json::to_string(&valid_open_ids).unwrap_or_else(|_|
"[]".to_string());

    // If we cleaned any orphaned refs, update the database silently
    if final_open_ids_str != open_ids_str || final_active != active_letter_id {
        let tx = conn.transaction().map_err(|e| e.to_string())?;
        tx.execute(
            "UPDATE workspace_state
            SET open_letter_ids = ?, active_letter_id = ?, updated_at = ?
            WHERE id = 1",
            rusqlite::params![final_open_ids_str, final_active,
Utc::now().to_rfc3339()],
        ).map_err(|e| e.to_string())?;
        tx.commit().map_err(|e| e.to_string())?;
    }

    Ok(WorkspaceState {
        open_letter_ids: final_open_ids_str,
        active_letter_id: final_active,
        crawler_state,
    })
}

// =====
// Zustand Persist Key-Value Storage Adapters
// =====

#[tauri::command]
pub fn get_persist_value(
    state: tauri::State<AppState>,
    key: String,
) -> Result<Option<String>, String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;
    let val: Result<String, rusqlite::Error> = conn.query_row(
        "SELECT value FROM zustand_persist WHERE key = ?",
        [&key],
        |row| row.get(0),
    );

    match val {
        Ok(v) => Ok(Some(v)),
        Err(rusqlite::Error::QueryReturnedNoRows) => Ok(None),
        Err(e) => Err(format!("Database error on persist fetch: {}", e)),
    }
}

#[tauri::command]
pub fn set_persist_value(
    state: tauri::State<AppState>,
    key: String,
    value: String,
) -> Result<bool, String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;

```

```

    conn.execute(
        "INSERT OR REPLACE INTO zustand_persist (key, value) VALUES (?, ?)",
        rusqlite::params![key, value],
    ).map_err(|e| format!("Failed to set persist value: {}", e));

    Ok(true)
}

#[tauri::command]
pub fn delete_persist_value(
    state: tauri::State<AppState>,
    key: String,
) -> Result<bool, String> {
    let conn = state.db.lock().map_err(|e| e.to_string())?;
    conn.execute(
        "DELETE FROM zustand_persist WHERE key = ?",
        [&key],
    ).map_err(|e| format!("Failed to delete persist value: {}", e));

    Ok(true)
}

--- END_FILE ---

--- START_FILE: src-tauri/src/db.rs ---

use rusqlite::{Connection, Result};
use std::path::Path;

/// Initialize the SQLite database with WAL mode, foreign keys, and run migrations.
pub fn init_database(db_path: &Path) -> Result<Connection> {
    let conn = Connection::open(db_path)?;

    // Required pragmas (BACKEND_STRUCTURE.md § Storage Rules)
    conn.pragma_update(None, "journal_mode", "WAL")?;
    conn.pragma_update(None, "foreign_keys", "ON")?;

    run_migrations(&conn)?;

    Ok(conn)
}

/// Run sequential migrations based on PRAGMA user_version.
/// Pattern from BACKEND_STRUCTURE.md § Migration Strategy.
fn run_migrations(conn: &Connection) -> Result<()> {
    let version: i32 = conn.pragma_query_value(None, "user_version", |row| row.get(0))?;

    if version < 1 {
        conn.execute_batch(include_str!("migrations/001_initial.sql"))?;
        conn.pragma_update(None, "user_version", &1)?;
    }
}

```

```

    Ok(())
}

--- END_FILE ---

--- START_FILE: src-tauri/src/lib.rs ---

mod db;
mod state;
pub mod parser;
pub mod commands;

use state::AppState;
use tauri::Manager;

#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_dialog::init())
        .plugin(tauri_plugin_fs::init())
        .plugin(tauri_plugin_clipboard_manager::init())
        .invoke_handler(tauri::generate_handler![
            // Journal commands
            commands::journal::import_journal_file,
            commands::journal::get_import_status,
            commands::journal::remove_journal_source,
            commands::journal::get_thought_units,
            // Penpal commands
            commands::penpal::create_penpal,
            commands::penpal::get_penpals,
            commands::penpal::update_penpal,
            commands::penpal::add_correspondence,
            commands::penpal::get_correspondence,
            // Letter commands
            commands::letter::create_letter,
            commands::letter::save_letter_content,
            commands::letter::load_letter_content,
            commands::letter::mark_letter_sent,
            // Workspace commands
            commands::workspace::save_workspace_state,
            commands::workspace::load_workspace_state,
            commands::workspace::get_persist_value,
            commands::workspace::set_persist_value,
            commands::workspace::delete_persist_value,
        ])
        .setup(|app| {
            let app_data_dir = app
                .path()
                .app_local_data_dir()
                .expect("Failed to resolve app local data dir");

```

```

        // Ensure the data directory exists
        std::fs::create_dir_all(&app_data_dir)
            .expect("Failed to create app data directory");

        let db_path = app_data_dir.join("journal_penpal.db");
        let conn = db::init_database(&db_path)
            .expect("Failed to initialize database");

        app.manage(AppState::new(conn));

        Ok(())
    })
    .run(tauri::generate_context!())
    .expect("error while running tauri application");
}

```

--- END_FILE ---

--- START_FILE: src-tauri/src/main.rs ---

```

#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]

```

```

fn main() {
    journal_to_penpal_pipeline_lib::run()
}

```

--- END_FILE ---

--- START_FILE: src-tauri/src/migrations/001_initial.sql ---

```

-- Migration 001: Initial schema
-- Source: BACKEND_STRUCTURE.md – all 7 tables + zustand_persist
-- Applied when PRAGMA user_version < 1

```

```

-- =====
-- Table: journal_sources
-- =====

```

```

CREATE TABLE IF NOT EXISTS journal_sources (
    id            TEXT PRIMARY KEY,
    file_path     TEXT NOT NULL UNIQUE,
    file_name     TEXT NOT NULL,
    last_modified TEXT NOT NULL,
    last_imported TEXT NOT NULL,
    format_version INTEGER NOT NULL,
    entry_count   INTEGER NOT NULL DEFAULT 0
);

```

```

CREATE INDEX IF NOT EXISTS idx_journal_sources_path ON journal_sources(file_path);

```

```

-- =====
-- Table: thought_units
-- =====
CREATE TABLE IF NOT EXISTS thought_units (
    id                TEXT PRIMARY KEY,
    date              TEXT,
    content           TEXT NOT NULL,
    category          TEXT NOT NULL DEFAULT 'uncategorized'
                                CHECK(category IN ('presence', 'reminiscence',
'uncategorized'))),
    source_file_path  TEXT NOT NULL,
    source_line_number INTEGER NOT NULL,
    format_version    INTEGER NOT NULL,
    created_at        TEXT NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_thought_units_date ON thought_units(date);
CREATE INDEX IF NOT EXISTS idx_thought_units_source ON thought_units(source_file_path);

-- =====
-- Table: penpals
-- =====
CREATE TABLE IF NOT EXISTS penpals (
    id                TEXT PRIMARY KEY,
    name              TEXT NOT NULL,
    country           TEXT NOT NULL DEFAULT '',
    interests         TEXT NOT NULL DEFAULT '',
    topics            TEXT NOT NULL DEFAULT '',
    notes             TEXT NOT NULL DEFAULT '',
    created_at        TEXT NOT NULL,
    updated_at        TEXT NOT NULL
);

-- =====
-- Table: correspondence
-- =====
CREATE TABLE IF NOT EXISTS correspondence (
    id                TEXT PRIMARY KEY,
    penpal_id         TEXT NOT NULL,
    direction         TEXT NOT NULL CHECK(direction IN ('sent', 'received')),
    content           TEXT NOT NULL,
    letter_date       TEXT NOT NULL,
    imported_at       TEXT NOT NULL,
    FOREIGN KEY (penpal_id) REFERENCES penpals(id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_correspondence_penpal ON correspondence(penpal_id);

-- =====
-- Table: letters
-- =====
CREATE TABLE IF NOT EXISTS letters (

```

```

    id            TEXT PRIMARY KEY,
    penpal_id     TEXT NOT NULL,
    status        TEXT NOT NULL DEFAULT 'draft' CHECK(status IN ('draft', 'sent')),
    created_at    TEXT NOT NULL,
    updated_at    TEXT NOT NULL,
    FOREIGN KEY (penpal_id) REFERENCES penpals(id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_letters_penpal ON letters(penpal_id);
CREATE INDEX IF NOT EXISTS idx_letters_status ON letters(status);

-- =====
-- Table: letter_content
-- =====
CREATE TABLE IF NOT EXISTS letter_content (
    id            TEXT PRIMARY KEY,
    letter_id     TEXT NOT NULL UNIQUE,
    blocks_json   TEXT NOT NULL DEFAULT '[]',
    FOREIGN KEY (letter_id) REFERENCES letters(id) ON DELETE CASCADE
);

CREATE UNIQUE INDEX IF NOT EXISTS idx_letter_content_letter ON
letter_content(letter_id);

-- =====
-- Table: workspace_state
-- =====
CREATE TABLE IF NOT EXISTS workspace_state (
    id            INTEGER PRIMARY KEY CHECK(id = 1),
    open_letter_ids TEXT NOT NULL DEFAULT '[]',
    active_letter_id TEXT,
    crawler_state  TEXT NOT NULL DEFAULT '{}',
    updated_at    TEXT NOT NULL
);

-- Insert the singleton row
INSERT OR IGNORE INTO workspace_state (id, open_letter_ids, active_letter_id,
crawler_state, updated_at)
VALUES (1, '[]', NULL, '{}', datetime('now'));

-- =====
-- Table: zustand_persist
-- =====
CREATE TABLE IF NOT EXISTS zustand_persist (
    key          TEXT PRIMARY KEY,
    value        TEXT NOT NULL
);

--- END_FILE ---

```



```
--- START_FILE: src-tauri/src/parser/format1.rs ---
```

```
use nom::{
    bytes::complete::tag,
    character::complete::{line_ending, not_line_ending, space0, anychar},
    combinator::opt,
    sequence::tuple,
    IResult, Parser,
};
use nom_locate::LocatedSpan;
use uuid::Uuid;
use super::{parse_date, ThoughtUnit, FormatVersion};

type Span<'a> = LocatedSpan<&'a str>;

#[derive(Debug)]
struct RawDateBlock {
    date_str: String,
    #[allow(dead_code)]
    date_line: i32,
    lines: Vec<(String, i32)>,
}

fn parse_date_header(input: Span) -> IResult<Span, (String, i32)> {
    let line_num = input.location_line() as i32;
    let (input, _) = tuple((tag("##"), space0)).parse(input)?;
    let (input, date_span) = not_line_ending(input)?;
    let (input, _) = opt(line_ending).parse(input)?;
    Ok((input, (date_span.fragment().trim().to_string(), line_num)))
}

fn parse_content_line(input: Span) -> IResult<Span, (Span, i32)> {
    let line_num = input.location_line() as i32;
    if input.fragment().starts_with("##") {
        return Err(nom::Err::Error(nom::error::Error::new(input,
nom::error::ErrorKind::Tag)));
    }
    let (input, line_content) = not_line_ending(input)?;
    let (input, _) = opt(line_ending).parse(input)?;
    Ok((input, (line_content, line_num)))
}

fn parse_pre_header_block(input: Span) -> IResult<Span, Vec<(String, i32)>> {
    let mut lines = Vec::new();
    let mut remaining = input;
    while !remaining.fragment().is_empty() {
        if remaining.fragment().starts_with("##") {
            break;
        }
        match parse_content_line(remaining) {
            Ok((next_input, (line_content, line_num))) => {
                let content_str = line_content.fragment().trim().to_string();
```

```

        if !content_str.is_empty() {
            lines.push((content_str, line_num));
        }
        remaining = next_input;
    }
    Err(_) => {
        let (next_input, _) = opt(line_ending).parse(remaining)?;
        if next_input == remaining {
            let (next_input, _) = anychar(remaining)?;
            remaining = next_input;
        } else {
            remaining = next_input;
        }
    }
}
Ok((remaining, lines))
}

```

```

fn parse_date_block(input: Span) -> IResult<Span, RawDateBlock> {
    let (input, (date_str, date_line)) = parse_date_header(input)?;
    let mut lines = Vec::new();
    let mut remaining = input;
    while !remaining.fragment().is_empty() {
        if remaining.fragment().starts_with("##") {
            break;
        }
        match parse_content_line(remaining) {
            Ok((next_input, (line_content, line_num))) => {
                let content_str = line_content.fragment().trim().to_string();
                if !content_str.is_empty() {
                    lines.push((content_str, line_num));
                }
                remaining = next_input;
            }
            Err(_) => {
                let (next_input, _) = opt(line_ending).parse(remaining)?;
                if next_input == remaining {
                    let (next_input, _) = anychar(remaining)?;
                    remaining = next_input;
                } else {
                    remaining = next_input;
                }
            }
        }
    }
    Ok((remaining, RawDateBlock { date_str, date_line, lines }))
}

```

```

pub fn parse(content: &str, file_path: &str, default_date: Option<&str>) ->
Vec<ThoughtUnit> {
    let mut thought_units = Vec::new();
    let created_at = chrono::Local::now().to_rfc3339();

```

```

let span = Span::new(content);

// 1. Parse pre-header block (if any)
let mut current_date = default_date.map(String::from);
if let Ok((remaining, pre_lines)) = parse_pre_header_block(span) {
    for (line_str, line_num) in pre_lines {
        if let Some(clean) = clean_bullet(&line_str) {
            thought_units.push(ThoughtUnit {
                id: Uuid::new_v4().to_string(),
                date: current_date.clone(),
                content: clean,
                category: "uncategorized".to_string(),
                source_file_path: file_path.to_string(),
                source_line_number: line_num,
                format_version: FormatVersion::Format1 as i32,
                created_at: created_at.clone(),
            });
        }
    }
}

// 2. Parse sequential date blocks
let mut rem = remaining;
while !rem.fragment().is_empty() {
    if rem.fragment().starts_with("##") {
        if let Ok((next_rem, block)) = parse_date_block(rem) {
            if let Some(parsed_d) = parse_date(&block.date_str) {
                current_date = Some(parsed_d.format("%Y-%m-%d").to_string());
            }

            for (line_str, line_num) in block.lines {
                if let Some(clean) = clean_bullet(&line_str) {
                    thought_units.push(ThoughtUnit {
                        id: Uuid::new_v4().to_string(),
                        date: current_date.clone(),
                        content: clean,
                        category: "uncategorized".to_string(),
                        source_file_path: file_path.to_string(),
                        source_line_number: line_num,
                        format_version: FormatVersion::Format1 as i32,
                        created_at: created_at.clone(),
                    });
                }
            }
            rem = next_rem;
        } else {
            // Safe break if it fails
            break;
        }
    } else {
        // Consume blank lines/junk
        if let Ok((next_rem, _)) = anychar::<Span, nom::error::Error<Span>>(rem)
    {
        rem = next_rem;
    }
}

```

```

        } else {
            break;
        }
    }
}

thought_units
}

fn clean_bullet(line: &str) -> Option<String> {
    let trimmed = line.trim();
    if trimmed.is_empty() {
        return None;
    }
    let mut clean = trimmed;
    if trimmed.starts_with("- ") {
        clean = trimmed.strip_prefix("- ").unwrap().trim();
    } else if trimmed.starts_with("* ") {
        clean = trimmed.strip_prefix("* ").unwrap().trim();
    } else if trimmed.starts_with("• ") {
        clean = trimmed.strip_prefix("• ").unwrap().trim();
    }

    if clean.is_empty() {
        None
    } else {
        Some(clean.to_string())
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_format1_parser() {
        let content = "Some initial line\n## 2024-05-12\n- Bullet 1\n* Bullet 2\nNormal text\n## May 13, 2024\n• Bullet 3";
        let units = parse(content, "test.md", Some("2024-05-11"));

        assert_eq!(units.len(), 5);
        assert_eq!(units[0].content, "Some initial line");
        assert_eq!(units[0].date, Some("2024-05-11".to_string()));
        assert_eq!(units[0].source_line_number, 1);

        assert_eq!(units[1].content, "Bullet 1");
        assert_eq!(units[1].date, Some("2024-05-12".to_string()));
        assert_eq!(units[1].source_line_number, 3);

        assert_eq!(units[2].content, "Bullet 2");
        assert_eq!(units[2].date, Some("2024-05-12".to_string()));
        assert_eq!(units[2].source_line_number, 4);
    }
}

```

```

        assert_eq!(units[3].content, "Normal text");
        assert_eq!(units[3].date, Some("2024-05-12".to_string()));
        assert_eq!(units[3].source_line_number, 5);

        assert_eq!(units[4].content, "Bullet 3");
        assert_eq!(units[4].date, Some("2024-05-13".to_string()));
        assert_eq!(units[4].source_line_number, 7);
    }
}

--- END_FILE ---

--- START_FILE: src-tauri/src/parser/format2.rs ---

use nom::{
    bytes::complete::tag,
    character::complete::{line_ending, not_line_ending, space0, anychar},
    combinator::opt,
    sequence::tuple,
    IResult, Parser,
};
use nom_locate::LocatedSpan;
use uuid::Uuid;
use super::{parse_date, ThoughtUnit, FormatVersion};

type Span<'a> = LocatedSpan<&'a str>;

#[derive(Debug)]
struct RawDateBlock {
    date_str: String,
    #[allow(dead_code)]
    date_line: i32,
    lines: Vec<(String, i32)>,
}

fn parse_date_header(input: Span) -> IResult<Span, (String, i32)> {
    let line_num = input.location_line() as i32;
    let (input, _) = tuple((tag("##"), space0)).parse(input)?;
    let (input, date_span) = not_line_ending(input)?;
    let (input, _) = opt(line_ending).parse(input)?;
    Ok((input, (date_span.fragment().trim().to_string(), line_num)))
}

fn parse_content_line(input: Span) -> IResult<Span, (Span, i32)> {
    let line_num = input.location_line() as i32;
    if input.fragment().starts_with("##") {
        return Err(nom::Err::Error(nom::error::Error::new(input,
nom::error::ErrorKind::Tag)));
    }
    let (input, line_content) = not_line_ending(input)?;

```

```

    let (input, _) = opt(line_ending).parse(input)?;
    Ok((input, (line_content, line_num)))
}

fn parse_pre_header_block(input: Span) -> IResult<Span, Vec<(String, i32)>> {
    let mut lines = Vec::new();
    let mut remaining = input;
    while !remaining.fragment().is_empty() {
        if remaining.fragment().starts_with("##") {
            break;
        }
        match parse_content_line(remaining) {
            Ok((next_input, (line_content, line_num))) => {
                let content_str = line_content.fragment().trim().to_string();
                if !content_str.is_empty() {
                    lines.push((content_str, line_num));
                }
                remaining = next_input;
            }
            Err(_) => {
                let (next_input, _) = opt(line_ending).parse(remaining)?;
                if next_input == remaining {
                    let (next_input, _) = anychar(remaining)?;
                    remaining = next_input;
                } else {
                    remaining = next_input;
                }
            }
        }
    }
    Ok((remaining, lines))
}

fn parse_date_block(input: Span) -> IResult<Span, RawDateBlock> {
    let (input, (date_str, date_line)) = parse_date_header(input)?;
    let mut lines = Vec::new();
    let mut remaining = input;
    while !remaining.fragment().is_empty() {
        if remaining.fragment().starts_with("##") {
            break;
        }
        match parse_content_line(remaining) {
            Ok((next_input, (line_content, line_num))) => {
                let content_str = line_content.fragment().trim().to_string();
                if !content_str.is_empty() {
                    lines.push((content_str, line_num));
                }
                remaining = next_input;
            }
            Err(_) => {
                let (next_input, _) = opt(line_ending).parse(remaining)?;
                if next_input == remaining {
                    let (next_input, _) = anychar(remaining)?;

```

```

        remaining = next_input;
    } else {
        remaining = next_input;
    }
}
}
}
Ok((remaining, RawDateBlock { date_str, date_line, lines }))
}

pub fn parse(content: &str, file_path: &str, default_date: Option<&str>) ->
Vec<ThoughtUnit> {
    let mut thought_units = Vec::new();
    let created_at = chrono::Local::now().to_rfc3339();
    let span = Span::new(content);

    // 1. Parse pre-header block (if any)
    let mut current_date = default_date.map(String::from);
    if let Ok((remaining, pre_lines)) = parse_pre_header_block(span) {
        for (line_str, line_num) in pre_lines {
            let trimmed = line_str.trim();
            if !trimmed.is_empty() {
                thought_units.push(ThoughtUnit {
                    id: Uuid::new_v4().to_string(),
                    date: current_date.clone(),
                    content: trimmed.to_string(),
                    category: "uncategorized".to_string(),
                    source_file_path: file_path.to_string(),
                    source_line_number: line_num,
                    format_version: FormatVersion::Format2 as i32,
                    created_at: created_at.clone(),
                });
            }
        }
    }

    // 2. Parse sequential date blocks
    let mut rem = remaining;
    while !rem.fragment().is_empty() {
        if rem.fragment().starts_with("##") {
            if let Ok((next_rem, block)) = parse_date_block(rem) {
                if let Some(parsed_d) = parse_date(&block.date_str) {
                    current_date = Some(parsed_d.format("%Y-%m-%d").to_string());
                }

                for (line_str, line_num) in block.lines {
                    let trimmed = line_str.trim();
                    if !trimmed.is_empty() {
                        thought_units.push(ThoughtUnit {
                            id: Uuid::new_v4().to_string(),
                            date: current_date.clone(),
                            content: trimmed.to_string(),
                            category: "uncategorized".to_string(),
                            source_file_path: file_path.to_string(),

```

```

        source_line_number: line_num,
        format_version: FormatVersion::Format2 as i32,
        created_at: created_at.clone(),
    });
    }
    }
    rem = next_rem;
} else {
    break;
}
} else {
    if let Ok((next_rem, _)) = anychar::<Span, nom::error::Error<Span>>(rem)
{
        rem = next_rem;
    } else {
        break;
    }
}
}

thought_units
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_format2_parser() {
        let content = "Some initial line\n## 2024-05-12\n- Keep bullet literal
here\nNormal paragraph line\n## May 13, 2024\nPlain newline content";
        let units = parse(content, "test.md", Some("2024-05-11"));

        assert_eq!(units.len(), 4);
        assert_eq!(units[0].content, "Some initial line");
        assert_eq!(units[0].date, Some("2024-05-11".to_string()));

        assert_eq!(units[1].content, "- Keep bullet literal here");
        assert_eq!(units[1].date, Some("2024-05-12".to_string()));

        assert_eq!(units[2].content, "Normal paragraph line");

        assert_eq!(units[3].content, "Plain newline content");
        assert_eq!(units[3].date, Some("2024-05-13".to_string()));
    }
}

--- END_FILE ---

```



```
--- START_FILE: src-tauri/src/parser/format3.rs ---
```

```
use nom::{
    bytes::complete::tag,
    character::complete::{line_ending, not_line_ending, space0, anychar},
    combinator::opt,
    sequence::tuple,
    IResult, Parser,
};
use nom_locate::LocatedSpan;
use uuid::Uuid;
use super::{parse_date, ThoughtUnit, FormatVersion};

type Span<'a> = LocatedSpan<&'a str>;

#[derive(Debug)]
struct RawDateBlock {
    date_str: String,
    #[allow(dead_code)]
    date_line: i32,
    lines: Vec<(String, i32)>,
}

fn parse_date_header(input: Span) -> IResult<Span, (String, i32)> {
    let line_num = input.location_line() as i32;
    let (input, _) = tuple((tag("##"), space0)).parse(input)?;
    let (input, date_span) = not_line_ending(input)?;
    let (input, _) = opt(line_ending).parse(input)?;
    Ok((input, (date_span.fragment().trim().to_string(), line_num)))
}

fn parse_content_line(input: Span) -> IResult<Span, (Span, i32)> {
    let line_num = input.location_line() as i32;
    if input.fragment().starts_with("##") {
        return Err(nom::Err::Error(nom::error::Error::new(input,
nom::error::ErrorKind::Tag)));
    }
    let (input, line_content) = not_line_ending(input)?;
    let (input, _) = opt(line_ending).parse(input)?;
    Ok((input, (line_content, line_num)))
}

fn parse_pre_header_block(input: Span) -> IResult<Span, Vec<(String, i32)>> {
    let mut lines = Vec::new();
    let mut remaining = input;
    while !remaining.fragment().is_empty() {
        if remaining.fragment().starts_with("##") {
            break;
        }
        match parse_content_line(remaining) {
            Ok((next_input, (line_content, line_num))) => {
                let content_str = line_content.fragment().trim().to_string();
```

```

        if !content_str.is_empty() {
            lines.push((content_str, line_num));
        }
        remaining = next_input;
    }
    Err(_) => {
        let (next_input, _) = opt(line_ending).parse(remaining)?;
        if next_input == remaining {
            let (next_input, _) = anychar(remaining)?;
            remaining = next_input;
        } else {
            remaining = next_input;
        }
    }
}
Ok((remaining, lines))
}

```

```

fn parse_date_block(input: Span) -> IResult<Span, RawDateBlock> {
    let (input, (date_str, date_line)) = parse_date_header(input)?;
    let mut lines = Vec::new();
    let mut remaining = input;
    while !remaining.fragment().is_empty() {
        if remaining.fragment().starts_with("##") {
            break;
        }
        match parse_content_line(remaining) {
            Ok((next_input, (line_content, line_num))) => {
                let content_str = line_content.fragment().trim().to_string();
                if !content_str.is_empty() {
                    lines.push((content_str, line_num));
                }
                remaining = next_input;
            }
            Err(_) => {
                let (next_input, _) = opt(line_ending).parse(remaining)?;
                if next_input == remaining {
                    let (next_input, _) = anychar(remaining)?;
                    remaining = next_input;
                } else {
                    remaining = next_input;
                }
            }
        }
    }
    Ok((remaining, RawDateBlock { date_str, date_line, lines }))
}

```

```

pub fn parse(content: &str, file_path: &str, default_date: Option<&str>) ->
Vec<ThoughtUnit> {
    let mut thought_units = Vec::new();
    let created_at = chrono::Local::now().to_rfc3339();

```

```

let span = Span::new(content);

let mut current_date = default_date.map(String::from);
let mut current_category = "uncategorized".to_string();

// Helper to process lines
let process_line = |line_str: &str, line_num: i32, date: &Option<String>, units:
&mut Vec<ThoughtUnit>, cat: &mut String| {
    let trimmed = line_str.trim();
    if trimmed == "#presence" {
        *cat = "presence".to_string();
    } else if trimmed == "#reminiscence" {
        *cat = "reminiscence".to_string();
    } else if !trimmed.is_empty() {
        units.push(ThoughtUnit {
            id: Uuid::new_v4().to_string(),
            date: date.clone(),
            content: trimmed.to_string(),
            category: cat.clone(),
            source_file_path: file_path.to_string(),
            source_line_number: line_num,
            format_version: FormatVersion::Format3 as i32,
            created_at: created_at.clone(),
        });
    }
};

// 1. Parse pre-header block (if any)
if let Ok((remaining, pre_lines)) = parse_pre_header_block(span) {
    for (line_str, line_num) in pre_lines {
        process_line(&line_str, line_num, &current_date, &mut thought_units, &mut
current_category);
    }

    // 2. Parse sequential date blocks
    let mut rem = remaining;
    while !rem.fragment().is_empty() {
        if rem.fragment().starts_with("##") {
            if let Ok((next_rem, block)) = parse_date_block(rem) {
                if let Some(parsed_d) = parse_date(&block.date_str) {
                    current_date = Some(parsed_d.format("%Y-%m-%d").to_string());
                }

                // Reset category to uncategorized for each new date block to avoid
bleeding state
                current_category = "uncategorized".to_string();

                for (line_str, line_num) in block.lines {
                    process_line(&line_str, line_num, &current_date, &mut
thought_units, &mut current_category);
                }
                rem = next_rem;
            } else {

```

```

        break;
    }
    } else {
        if let Ok((next_rem, _)) = anychar::<Span, nom::error::Error<Span>>(rem)
{
            rem = next_rem;
        } else {
            break;
        }
    }
}

thought_units
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_format3_parser() {
        let content = "Some initial line\n## 2024-05-12\n#presence\nPresence item
1\n#reminiscence\nReminiscence item 1\n## May 13, 2024\nUntagged
item\n#presence\nPresence item 2";
        let units = parse(content, "test.md", Some("2024-05-11"));

        assert_eq!(units.len(), 5);
        assert_eq!(units[0].content, "Some initial line");
        assert_eq!(units[0].category, "uncategorized");
        assert_eq!(units[0].date, Some("2024-05-11".to_string()));

        assert_eq!(units[1].content, "Presence item 1");
        assert_eq!(units[1].category, "presence");
        assert_eq!(units[1].date, Some("2024-05-12".to_string()));

        assert_eq!(units[2].content, "Reminiscence item 1");
        assert_eq!(units[2].category, "reminiscence");
        assert_eq!(units[2].date, Some("2024-05-12".to_string()));

        assert_eq!(units[3].content, "Untagged item");
        assert_eq!(units[3].category, "uncategorized"); // category reset to
uncategorized on new date block
        assert_eq!(units[3].date, Some("2024-05-13".to_string()));

        assert_eq!(units[4].content, "Presence item 2");
        assert_eq!(units[4].category, "presence");
        assert_eq!(units[4].date, Some("2024-05-13".to_string()));
    }
}

--- END_FILE ---

```

```
--- START_FILE: src-tauri/src/parser/mod.rs ---
```

```
pub mod format1;  
pub mod format2;  
pub mod format3;
```

```
use chrono::NaiveDate;  
use serde::{Deserialize, Serialize};  
use std::path::Path;  
use uuid::Uuid;
```

```
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq, Eq)]
```

```
#[serde(rename_all = "camelCase")]
```

```
pub struct ThoughtUnit {  
    pub id: String,  
    pub date: Option<String>, // ISO date "YYYY-MM-DD"  
    pub content: String,  
    pub category: String, // "presence" | "reminiscence" | "uncategorized"  
    pub source_file_path: String,  
    pub source_line_number: i32,  
    pub format_version: i32,  
    pub created_at: String,  
}
```

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
```

```
pub enum FormatVersion {  
    Format1 = 1,  
    Format2 = 2,  
    Format3 = 3,  
}
```

```
/// Helper function to detect the format variant of a journal file.
```

```
/// Score against each format pattern based on research findings:
```

```
/// - Format 3 (newest): Look for "#presence" or "#reminiscence" tags.
```

```
/// - Format 1 (oldest): Look for bullet markers ("- ", "* ", "• ") starting lines.
```

```
/// - Format 2 (middle): Default fallback (newline-delimited, date headers only).
```

```
pub fn detect_format(content: &str) -> FormatVersion {  
    if content.contains("#presence") || content.contains("#reminiscence") {  
        FormatVersion::Format3  
    } else if content.lines().any(|l| {  
        let trimmed = l.trim_start();  
        trimmed.starts_with("- ") || trimmed.starts_with("* ") || trimmed.starts_with("• ")  
    }) {  
        FormatVersion::Format1  
    } else {  
        FormatVersion::Format2  
    }  
}
```

```
/// Helper function to parse dates using chrono with multiple formats.
```

```
pub fn parse_date(date_str: &str) -> Option<NaiveDate> {
```

```

let normalized = date_str.trim();
let chars: Vec<char> = normalized.chars().collect();
let mut normalized_str = String::new();
let mut i = 0;
while i < chars.len() {
    if i + 2 < chars.len()
        && chars[i] == ' '
        && chars[i+1].is_ascii_digit()
        && (chars[i+2] == ',' || chars[i+2] == ' ')
        && (i == 0 || chars[i-1] != ' ')
    {
        normalized_str.push(' ');
        normalized_str.push('0');
        normalized_str.push(chars[i+1]);
        i += 2;
    } else {
        normalized_str.push(chars[i]);
        i += 1;
    }
}

let formats = [
    "%Y-%m-%d",
    "%B %d, %Y",
    "%d/%m/%Y",
    "%d %B %Y",
    "%B %d, %a, %Y",
    "%B %e, %a, %Y",
];
formats.iter()
    .find_map(|fmt| NaiveDate::parse_from_str(normalized_str.trim(), fmt).ok())
}

/// Helper function to extract a date from a filename (e.g., "journal-2024-05-12.md" ->
"2024-05-12").
pub fn extract_date_from_filename(file_path: &str) -> Option<String> {
    let path = Path::new(file_path);
    let filename = path.file_name()?.to_str()?;

    // Scan for YYYY-MM-DD pattern
    for i in 0..=filename.len().checked_sub(10).unwrap_or(0) {
        if let Some(sub) = filename.get(i..i+10) {
            // Ensure the match starts with a digit to avoid parsing leading hyphens as
            negative years (e.g. "-2024-05-1")
            if sub.chars().next().map_or(false, |c| c.is_ascii_digit()) {
                if let Ok(date) = NaiveDate::parse_from_str(sub, "%Y-%m-%d") {
                    return Some(date.format("%Y-%m-%d").to_string());
                }
            }
        }
    }
}
None
}

```

```

/// Master function to parse any journal file.
/// Detects format and routes to the appropriate parser.
/// If parsing fails or yields no dated entries, fallback is triggered.
pub fn parse_journal(
    content: &str,
    file_path: &str,
) -> (Vec<ThoughtUnit>, FormatVersion, Vec<String>) {
    let mut file_date = None;
    let mut parsed_content = content;

    if let Some(first_line) = content.lines().next() {
        let trimmed = first_line.trim();
        if trimmed.starts_with("### ") {
            let date_candidate = trimmed["### ".len()..].trim();
            if let Some(parsed) = parse_date(date_candidate) {
                file_date = Some(parsed.format("%Y-%m-%d").to_string());
                if let Some(pos) = content.find('\n') {
                    parsed_content = &content[pos + 1..];
                } else {
                    parsed_content = "";
                }
            }
        }
    }

    let file_date = file_date.or_else(|| extract_date_from_filename(file_path));
    let format = detect_format(parsed_content);

    let mut warnings = Vec::new();
    let mut thought_units = match format {
        FormatVersion::Format3 => format3::parse(parsed_content, file_path,
file_date.as_deref()),
        FormatVersion::Format1 => format1::parse(parsed_content, file_path,
file_date.as_deref()),
        FormatVersion::Format2 => format2::parse(parsed_content, file_path,
file_date.as_deref()),
    };

    // If the parsed count is 0 and it has date headers, or if we have general content
    // we can attempt a Best-Effort unstructured fallback if we didn't get any entries.
    if thought_units.is_empty() && !parsed_content.trim().is_empty() {
        // Fallback: treat each non-empty line as uncategorized thought-unit
        let created_at = chrono::Local::now().to_rfc3339();
        for (i, line) in parsed_content.lines().enumerate() {
            let line_content = line.trim();
            if !line_content.is_empty() {
                thought_units.push(ThoughtUnit {
                    id: Uuid::new_v4().to_string(),
                    date: file_date.clone(),
                    content: line_content.to_string(),
                    category: "uncategorized".to_string(),
                    source_file_path: file_path.to_string(),
                })
            }
        }
    }
}

```

```

        source_line_number: (i + 1) as i32,
        format_version: format as i32,
        created_at: created_at.clone(),
    });
}
}
warnings.push(format!(
    "File '{}' parsed as unstructured fallback – some entries may be missing
dates.",
    Path::new(file_path).file_name().unwrap_or_default().to_string_lossy()
));
}

// Count undated entries to warn user (PRD criteria)
let undated_count = thought_units.iter().filter(|tu| tu.date.is_none()).count();
if undated_count > 0 && !thought_units.is_empty() {
    warnings.push(format!(
        "{} entries could not be dated.",
        undated_count
    ));
}

(thought_units, format, warnings)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_detect_format() {
        let f3_content = "## 2024-05-12\n#presence\nSome thought\n#reminiscence\nAnother
thought";
        assert_eq!(detect_format(f3_content), FormatVersion::Format3);

        let f1_content = "## 2024-05-12\n- A bullet point\n* Another bullet";
        assert_eq!(detect_format(f1_content), FormatVersion::Format1);

        let f2_content = "## 2024-05-12\nJust plain paragraph text\nNo bullet points";
        assert_eq!(detect_format(f2_content), FormatVersion::Format2);
    }

    #[test]
    fn test_parse_date() {
        assert_eq!(parse_date("2024-05-12"), Some(NaiveDate::from_ymd_opt(2024, 5,
12).unwrap()));
        assert_eq!(parse_date("May 12, 2024"), Some(NaiveDate::from_ymd_opt(2024, 5,
12).unwrap()));
        assert_eq!(parse_date("12/05/2024"), Some(NaiveDate::from_ymd_opt(2024, 5,
12).unwrap()));
        assert_eq!(parse_date("12 May 2024"), Some(NaiveDate::from_ymd_opt(2024, 5,
12).unwrap()));
        assert_eq!(parse_date("March 11, Mon, 2024"), Some(NaiveDate::from_ymd_opt(2024,

```



```

3, 11).unwrap()));
    assert_eq!(parse_date("March 9, Sat, 2024"), Some(NaiveDate::from_ymd_opt(2024,
3, 9).unwrap()));
    assert_eq!(parse_date("not a date"), None);
}

#[test]
fn test_extract_date_from_filename() {
    assert_eq!(
        extract_date_from_filename("C:/Users/USER/journal-2024-05-12.md"),
        Some("2024-05-12".to_string())
    );
    assert_eq!(
        extract_date_from_filename("journal_entry.txt"),
        None
    );
}

#[test]
fn test_extract_date_from_file_header() {
    let content_with_header = "### March 11, Mon, 2024\n* Woke up quite early,
played pes.";
    let (units, format, _warnings) = parse_journal(content_with_header,
"C:/Users/USER/journal.md");

    assert_eq!(format, FormatVersion::Format1);
    assert!(!units.is_empty());
    assert_eq!(units[0].date, Some("2024-03-11".to_string()));
    assert_eq!(units[0].content, "Woke up quite early, played pes.");

    // Also test single digit day equivalent
    let content_single_digit = "### March 9, Sat, 2024\n* Woke up quite early,
played pes.";
    let (units_sd, _, _) = parse_journal(content_single_digit,
"C:/Users/USER/journal.md");
    assert!(!units_sd.is_empty());
    assert_eq!(units_sd[0].date, Some("2024-03-09".to_string()));
}
}

--- END_FILE ---

--- START_FILE: src-tauri/src/state.rs ---

use rusqlite::Connection;
use std::sync::Mutex;

/// Application state holding the SQLite connection behind a Mutex.
/// All database access is serialized through this mutex – no race conditions
/// (see BACKEND_STRUCTURE.md § Edge Cases & Data Integrity).
pub struct AppState {

```

```

    pub db: Mutex<Connection>,
}

impl AppState {
    pub fn new(conn: Connection) -> Self {
        Self {
            db: Mutex::new(conn),
        }
    }
}

--- END_FILE ---

--- START_FILE: src/App.tsx ---

import { useState } from 'react';
import { CrawlerPanel } from '../components/CrawlerPanel';
import { WorkspacePanel } from '../components/WorkspacePanel';
import { PenpalPanel } from '../components/PenpalPanel';

function App() {
    const [showSources, setShowSources] = useState(false);

    return (
        <div className="app-shell">
            {/* 1. Left Column: Virtualized Crawler Panel */}
            <aside className="panel-left">
                <CrawlerPanel
                    onManageSources={() => setShowSources(true)}
                />
            </aside>

            {/* 2. Center Column: Tabbed Workspace Panel */}
            <main className="panel-center">
                <WorkspacePanel
                    showSources={showSources}
                    onCloseSources={() => setShowSources(false)}
                    onOpenSources={() => setShowSources(true)}
                />
            </main>

            {/* 3. Right Column: Penpals Management Panel */}
            <aside className="panel-right">
                <PenpalPanel />
            </aside>
        </div>
    );
}

export default App;

```

--- END_FILE ---

--- START_FILE: src/components/CrawlerPanel.tsx ---

```
import React, { useState, useEffect, useMemo, useRef } from 'react';
import { useVirtualizer } from '@tanstack/react-virtual';
import { useAppStore } from '../stores/appStore';
import { getThoughtUnits } from '../services/journalService';
import { ThoughtUnit } from '../types';
import { Search, Calendar, ChevronDown, Send, RefreshCw } from 'lucide-react';

interface CrawlerPanelProps {
  onManageSources: () => void;
}

export const CrawlerPanel: React.FC<CrawlerPanelProps> = ({ onManageSources }) => {
  const { crawlerFilters, setCrawlerFilters, openLetters, penpals, addToRoutingQueue } =
    useAppStore();
  const { startDate, endDate, category, searchQuery } = crawlerFilters;

  const [thoughtUnits, setThoughtUnits] = useState<ThoughtUnit[]>([]);
  const [loading, setLoading] = useState(false);
  const [routeMenuOpen, setRouteMenuOpen] = useState<string | null>(null);
  const [selectedLetters, setSelectedLetters] = useState<string[]>([]);

  // Reset selected letters when menu changes
  useEffect(() => {
    setSelectedLetters([]);
  }, [routeMenuOpen]);

  // Shuffle state
  const [isShuffled, setIsShuffled] = useState(false);
  const [shuffleSeed, setShuffleSeed] = useState(0);

  // 1. Fetch from database on filter change
  const refreshUnits = async () => {
    setLoading(true);
    try {
      const data = await getThoughtUnits(startDate, endDate, category);
      setThoughtUnits(data);
    } catch (e) {
      console.error('Failed to get thought units:', e);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    refreshUnits();
  }, [startDate, endDate, category]);
```

```

// 2. Client-side search substring matching
const filteredUnits = useMemo(() => {
  if (!searchQuery.trim()) return thoughtUnits;
  const q = searchQuery.toLowerCase();
  return thoughtUnits.filter((tu) => tu.content.toLowerCase().includes(q));
}, [thoughtUnits, searchQuery]);

// 2b. Shuffling active list
const displayedUnits = useMemo(() => {
  let units = [...filteredUnits];
  if (isShuffled) {
    // Fisher-Yates Shuffle
    for (let i = units.length - 1; i > 0; i--) {
      const j = Math.floor(Math.random() * (i + 1));
      const temp = units[i] as ThoughtUnit;
      units[i] = units[j] as ThoughtUnit;
      units[j] = temp;
    }
  }
  return units;
}, [filteredUnits, isShuffled, shuffleSeed]);

// 3. Virtualizer Setup
const parentRef = useRef<HTMLDivElement>(null);
const rowVirtualizer = useVirtualizer({
  count: displayedUnits.length,
  getScrollElement: () => parentRef.current,
  estimateSize: () => 110,
  overscan: 5,
});

// Map open letters to penpal names
const activeTabs = useMemo(() => {
  return openLetters.map((tab) => {
    const penpal = penpals.find((p) => p.id === tab.penpalId);
    return {
      letterId: tab.letterId,
      penpalName: penpal ? penpal.name : 'Unknown Penpal',
    };
  });
}, [openLetters, penpals]);

// Handle routing to multiple target letter tabs
const handleRouteMultiple = (tu: ThoughtUnit) => {
  if (selectedLetters.length === 0) return;

  let successCount = 0;
  let failedCount = 0;

  selectedLetters.forEach((targetLetterId) => {
    try {
      addToRoutingQueue({
        id: crypto.randomUUID(),

```

```

        sourceThoughtUnit: tu,
        targetLetterId,
        insertPosition: 'end',
        timestamp: Date.now(),
    });
    successCount++;
} catch (err: any) {
    console.error(`Failed to route to letter ${targetLetterId}:`, err);
    failedCount++;
}
});

if (failedCount === 0) {
    alert(`Successfully routed entry to ${successCount} draft(s)!`);
} else {
    alert(`Successfully routed to ${successCount} draft(s). Failed for ${failedCount} draft(s).`);
}

setRouteMenuOpen(null);
setSelectedLetters([]);
};

// Timeframe presets handler
const handlePreset = (preset: 'week' | 'month' | 'all') => {
    const today = new Date();
    const pad = (n: number) => n.toString().padStart(2, '0');
    const formatDate = (d: Date) => `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`;

    if (preset === 'week') {
        const start = new Date(today);
        const day = start.getDay();
        const diff = start.getDate() - day + (day === 0 ? -6 : 1);
        const monday = new Date(start.setDate(diff));
        setCrawlerFilters({
            startDate: formatDate(monday),
            endDate: formatDate(today)
        });
    } else if (preset === 'month') {
        const firstDay = new Date(today.getFullYear(), today.getMonth(), 1);
        setCrawlerFilters({
            startDate: formatDate(firstDay),
            endDate: formatDate(today)
        });
    } else {
        setCrawlerFilters({
            startDate: null,
            endDate: null
        });
    }
};

```

```

const getCategoryBadgeClass = (cat: string) => {
  switch (cat) {
    case 'presence':
      return 'chip-presence';
    case 'reminiscence':
      return 'chip-reminiscence';
    default:
      return 'chip-uncategorized';
  }
};

return (
  <div className="crawler-panel-container" style={{ display: 'flex', flexDirection:
'column', height: '100%', borderRight: '1px solid var(--color-border)' }}>
    /* Panel Title & Header Buttons */
    <div style={{ padding: 'var(--space-4)', borderBottom: '1px solid
var(--color-border)', display: 'flex', justifyContent: 'space-between', alignItems:
'center' }}>
      <div style={{ display: 'flex', alignItems: 'center', gap: 'var(--space-2)' }}>
        <h2 style={{ fontSize: 'var(--text-lg)', fontWeight: 'var(--weight-semibold)',
color: 'var(--color-text-primary)' }}>Journal Crawler</h2>
        {isShuffled && (
          <span style={{ fontSize: '10px', background: 'rgba(2, 132, 199, 0.1)',
color: 'var(--color-primary)', borderRadius: 'var(--radius-sm)', padding: '1px 6px',
fontWeight: 'var(--weight-semibold)' }}>
            Shuffled
          </span>
        )}
      </div>
      <div style={{ display: 'flex', gap: 'var(--space-2)' }}>
        <button
          onClick={() => {
            if (isShuffled) {
              setIsShuffled(false);
            } else {
              setIsShuffled(true);
              setShuffleSeed((prev) => prev + 1);
            }
          }}
          style={{
            background: isShuffled ? 'var(--color-primary)' :
'var(--color-surface-elevated)',
            color: isShuffled ? 'white' : 'var(--color-text-primary)',
            border: '1px solid var(--color-border)',
            padding: 'var(--space-1) var(--space-2)',
            borderRadius: 'var(--radius-md)',
            fontSize: 'var(--text-xs)',
            cursor: 'pointer',
            fontWeight: 'var(--weight-medium)',
            display: 'flex',
            alignItems: 'center',
            gap: 'var(--space-1)'
          }}
        </div>
    </div>

```

```

    >
        <RefreshCw size={12} className={isShuffled ? 'spin' : ''} style={{ width:
'12px', height: '12px' }} />
        Randomize
    </button>

    <button
        onClick={onManageSources}
        className="btn-sources"
        style={{
            background: 'var(--color-surface-elevated)',
            color: 'var(--color-primary)',
            border: '1px solid var(--color-border)',
            padding: 'var(--space-1) var(--space-2)',
            borderRadius: 'var(--radius-md)',
            fontSize: 'var(--text-xs)',
            cursor: 'pointer',
            fontWeight: 'var(--weight-medium)',
            display: 'flex',
            alignItems: 'center',
            gap: 'var(--space-1)'
        }}
    >
        Sources
    </button>
</div>
</div>

{/* Filters Form */}
    <div style={{ padding: 'var(--space-4)', borderBottom: '1px solid
var(--color-border)', display: 'flex', flexDirection: 'column', gap: 'var(--space-3)',
background: 'var(--color-background)' }}>
        {/* Search Input */}
        <div style={{ position: 'relative' }}>
            <Search size={14} style={{ position: 'absolute', left: 10, top: 10, color:
'var(--color-text-secondary)' }} />
            <input
                type="text"
                value={searchQuery}
                onChange={(e) => setCrawlerFilters({ searchQuery: e.target.value })}
                placeholder="Search entries..."
                style={{
                    width: '100%',
                    background: 'var(--color-surface)',
                    border: '1px solid var(--color-border)',
                    borderRadius: 'var(--radius-md)',
                    padding: 'var(--space-2) var(--space-3) var(--space-2) 32px',
                    color: 'var(--color-text-primary)',
                    fontSize: 'var(--text-sm)',
                }}
            />
        </div>
    </div>

```

```

    { /* Date Ranges */ }
    <div style={{ display: 'flex', gap: 'var(--space-2)' }}>
      <div style={{ flex: 1, position: 'relative' }}>
        <Calendar size={12} style={{ position: 'absolute', left: 8, top: 10, color:
'var(--color-text-secondary)' }} />
        <input
          type="date"
          value={startDate || ''}
          onChange={(e) => setCrawlerFilters({ startDate: e.target.value || null })}
          style={{
            width: '100%',
            background: 'var(--color-surface)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-md)',
            padding: 'var(--space-2) var(--space-2) var(--space-2) 26px',
            color: 'var(--color-text-primary)',
            fontSize: 'var(--text-xs)',
          }}
        />
      </div>
      <div style={{ flex: 1, position: 'relative' }}>
        <Calendar size={12} style={{ position: 'absolute', left: 8, top: 10, color:
'var(--color-text-secondary)' }} />
        <input
          type="date"
          value={endDate || ''}
          onCh
ange={(e) => setCrawlerFilters({ endDate: e.target.value || null })}
          style={{
            width: '100%',
            background: 'var(--color-surface)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-md)',
            padding: 'var(--space-2) var(--space-2) var(--space-2) 26px',
            color: 'var(--color-text-primary)',
            fontSize: 'var(--text-xs)',
          }}
        />
      </div>
    </div>

    { /* Timeframe Presets */ }
    <div style={{ display: 'flex', gap: 'var(--space-2)' }}>
      {[['This Week', 'This Month', 'All Time'] as const].map((label) => {
        const presetKey = label === 'This Week' ? 'week' : label === 'This Month' ?
'month' : 'all';
        return (
          <button
            key={label}
            type="button"
            onClick={() => handlePreset(presetKey)}
            style={{
              flex: 1,

```



```

        background: 'var(--color-surface)',
        color: 'var(--color-text-primary)',
        border: '1px solid var(--color-border)',
        borderRadius: 'var(--radius-sm)',
        fontSize: '10px',
        padding: '4px 6px',
        cursor: 'pointer',
        fontWeight: 'var(--weight-medium)',
    }}
    onMouseOver={(e) => (e.currentTarget.style.borderColor =
'var(--color-primary)')}}
    onMouseOut={(e) => (e.currentTarget.style.borderColor =
'var(--color-border)')}}
    >
        {label}
    </button>
);
}}}
</div>

{/* Category Buttons */}
<div style={{ display: 'flex', gap: 'var(--space-2)' }}>
    {[('all', 'presence', 'reminiscence', 'uncategorized'] as const).map((cat) =>
{
    const isActive = cat === 'all' ? category === null : category === cat;
    return (
        <button
            key={cat}
            onClick={() => setCrawlerFilters({ category: cat === 'all' ? null : cat
        }}}
        style={{
            flex: 1,
            background: isActive ? 'var(--color-primary)' :
'var(--color-surface)',
            color: isActive ? 'var(--color-surface)' :
'var(--color-text-secondary)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-sm)',
            fontSize: 'var(--text-xs)',
            padding: 'var(--space-1) 0',
            cursor: 'pointer',
            fontWeight: 'var(--weight-semibold)',
            textTransform: 'capitalize',
        }}
        >
            {cat}
        </button>
    );
    }}}
</div>
</div>

{/* Main Stream Area */}

```

```

<div style={{ flex: 1, overflow: 'hidden', position: 'relative' }}>
  {loading && (
    <div style={{ position: 'absolute', inset: 0, background: 'rgba(20, 20, 32, 0.7)', display: 'flex', justifyContent: 'center', alignItems: 'center', zIndex: 10 }}>
      <RefreshCw size={24} className="spin" style={{ color: 'var(--color-primary)' }} />
    </div>
  )}

  {displayedUnits.length === 0 ? (
    <div style={{ padding: 'var(--space-8)', textAlign: 'center', color: 'var(--color-text-secondary)' }}>
      No journal entries match. Change filters or import files.
    </div>
  ) : (
    <div
      ref={parentRef}
      style={{
        height: '100%',
        overflow: 'auto',
        position: 'relative',
      }}
    >
      <div
        style={{
          height: `${rowVirtualizer.getTotalSize()}px`,
          width: '100%',
          position: 'relative',
        }}
      >
        {rowVirtualizer.getVirtualItems().map((virtualRow) => {
          const tu = displayedUnits[virtualRow.index];
          if (!tu) return null;
          const isMenuOpen = routeMenuOpen === tu.id;

          return (
            <div
              key={virtualRow.key}
              data-index={virtualRow.index}
              ref={rowVirtualizer.measureElement}
              style={{
                position: 'absolute',
                top: 0,
                left: 0,
                width: '100%',
                transform: `translateY(${virtualRow.start}px)`,
                padding: 'var(--space-2) var(--space-4)',
              }}
            >
              <div
                draggable
                onDragStart={(e) => {
                  e.dataTransfer.effectAllowed = 'copy';

```

```
e.dataTransfer.setData('application/x-thought-unit',
```

```
JSON.stringify(tu));
```

```
    e.dataTransfer.setData('text/plain', tu.content);
```

```
  }}
```

```
  style={{
```

```
    background: 'var(--color-surface-elevated)',
```

```
    border: '1px solid var(--color-border)',
```

```
    borderRadius: 'var(--radius-md)',
```

```
    padding: 'var(--space-3)',
```

```
    display: 'flex',
```

```
    flexDirection: 'column',
```

```
    gap: 'var(--space-2)',
```

```
    cursor: 'grab',
```

```
  }}
```

```
>
```

```
    <div style={{ display: 'flex', justifyContent: 'space-between',  
alignItems: 'center' }}>
```

```
      <span className={getCategoryBadgeClass(tu.category)}>
```

```
        #{tu.category}
```

```
      </span>
```

```
        <span style={{ fontSize: 'var(--text-xs)', color:  
'var(--color-text-secondary)' }}>
```

```
          {tu.date || 'Undated'}
```

```
        </span>
```

```
    </div>
```

```
    { /* Content */ }
```

```
      <div style={{ fontSize: 'var(--text-sm)', color:  
'var(--color-text-primary)', whiteSpace: 'pre-wrap', lineHeight: 'var(--leading-sm)' }}>  
        {tu.content}
```

```
      </div>
```

```
    { /* Line reference & Routing button */ }
```

```
      <div style={{ display: 'flex', justifyContent: 'space-between',  
alignItems: 'center', borderTop: '1px solid var(--color-border)', paddingTop:  
'var(--space-2)', marginTop: 'var(--space-1)' }}>
```

```
        <span style={{ fontSize: 'var(--text-xs)', color:  
'var(--color-text-secondary)' }}>
```

```
          Line {tu.sourceLineNumber}
```

```
        </span>
```

```
      <div style={{ position: 'relative' }}>
```

```
        <button
```

```
          onClick={() => setRouteMenuOpen(isMenuOpen ? null : tu.id)}
```

```
          style={{
```

```
            background: 'var(--color-surface)',
```

```
            color: 'var(--color-text-primary)',
```

```
            border: '1px solid var(--color-border)',
```

```
            borderRadius: 'var(--radius-sm)',
```

```
            padding: 'var(--space-1) var(--space-2)',
```

```
            fontSize: 'var(--text-xs)',
```

```
            cursor: 'pointer',
```

```
            display: 'flex',
```

```

        alignItems: 'center',
        gap: 'var(--space-1)',
    }}
>
<Send size={10} />
Route
<ChevronDown size={10} />
</button>

{isMenuOpen && (
    <div
        style={{
            position: 'absolute',
            right: 0,
            bottom: '100%',
            background: 'var(--color-surface-elevated)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-md)',
            boxShadow: 'var(--shadow-md)',
            zIndex: 100,
            minWidth: '180px',
            display: 'flex',
            flexDirection: 'column',
            overflow: 'hidden',
            marginBottom: 'var(--space-1)'
        }}
    >
        <div style={{ padding: 'var(--space-2) var(--space-3)',
fontSize: 'var(--text-xs)', fontWeight: 'var(--weight-semibold)', borderBottom: '1px
solid var(--color-border)', color: 'var(--color-text-secondary)' }}>
            Target Draft Tabs
        </div>
        {activeTabs.length === 0 ? (
            <div style={{ padding: 'var(--space-2) var(--space-3)',
fontSize: 'var(--text-xs)', color: 'var(--color-text-secondary)', fontStyle: 'italic'
}}>
                No draft letters open
            </div>
        ) : (
            <>
                <div style={{ display: 'flex', justifyContent:
'space-between', padding: '4px var(--space-3)', borderBottom: '1px solid
var(--color-border)', background: 'var(--color-background)' }}>
                    <button
                        type="button"
                        onClick={() =>
setSelectedLetters(activeTabs.map((t) => t.letterId))}
                        style={{ background: 'none', bo
rder: 'none', color: 'var(--color-primary)', fontSize: '10px', cursor: 'pointer',
padding: 0 }}
                    >
                        Select All
                    </button>
                </div>
            </>
        )
    </div>
)
}

```

```

        <button
          type="button"
          onClick={() => setSelectedLetters([])}
          style={{ background: 'none', border: 'none',
color: 'var(--color-text-secondary)', fontSize: '10px', cursor: 'pointer', padding: 0 }}
        >
          Clear
        </button>
      </div>
      <div style={{ maxHeight: '150px', overflowY: 'auto',
display: 'flex', flexDirection: 'column' }}>
        {activeTabs.map((tab) => {
          const isChecked =
selectedLetters.includes(tab.letterId);
          return (
            <label
              key={tab.letterId}
              style={{
                display: 'flex',
                alignItems: 'center',
                gap: 'var(--space-2)',
                padding: '6px var(--space-3)',
                fontSize: 'var(--text-xs)',
                color: 'var(--color-text-primary)',
                cursor: 'pointer',
                userSelect: 'none',
              }}
              onMouseOver={(e) =>
(e.currentTarget.style.background = 'var(--color-surface)')}
              onMouseOut={(e) =>
(e.currentTarget.style.background = 'none')}
            >
              <input
                type="checkbox"
                checked={isChecked}
                onChange={() => {
                  setSelectedLetters((prev) =>
                    isChecked
                      ? prev.filter((id) => id !==
tab.letterId)
                      : [...prev, tab.letterId]
                )};
              </input>
              {tab.penpalName}
            </label>
          );
        })}
      </div>
      <div style={{ padding: 'var(--space-2)', borderTop:
'1px solid var(--color-border)', display: 'flex' }}>
        <button

```

```

        type="button"
        disabled={selectedLetters.length === 0}
        onClick={() => handleRouteMultiple(tu)}
        style={{
            width: '100%',
            background: selectedLetters.length === 0 ?
'var(--color-text-disabled)' : 'var(--color-primary)',
            color: selectedLetters.length === 0 ?
'var(--color-text-secondary)' : 'white',
            border: 'none',
            borderRadius: 'var(--radius-sm)',
            padding: '6px 0',
            fontSize: '11px',
            fontWeight: 'var(--weight-semibold)',
            cursor: selectedLetters.length === 0 ?
'not-allowed' : 'pointer',
        }}
    >
        Confirm Route ({selectedLetters.length})
    </button>
</div>
</>
    )}
</div>
    )}
</div>
</div>
</div>
    );
    )}
</div>
</div>
    )}
</div>
</div>
    );
};

```

--- END_FILE ---

--- START_FILE: src/components/JournalSourcesView.tsx ---

```

import React, { useEffect, useState } from 'react';
import { open as openDialog } from '@tauri-apps/plugin-dialog';
import { getImportStatus, importJournalFile, removeJournalSource } from
'../services/journalService';
import { JournalSource } from '../types';
import { FileText, Plus, Trash2, RefreshCw, AlertTriangle, CheckCircle, Info } from
'luclide-react';

```

```

export const JournalSourcesView: React.FC = () => {
  const [sources, setSources] = useState<JournalSource[]>([]);
  const [loading, setLoading] = useState(false);
  const [warnings, setWarnings] = useState<string[]>([]);
  const [successMsg, setSuccessMsg] = useState<string | null>(null);

  const fetchSources = async () => {
    setLoading(true);
    try {
      const data = await getImportStatus();
      setSources(data);
    } catch (e) {
      console.error('Failed to fetch journal sources:', e);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchSources();
  }, []);

  const handleImport = async () => {
    setWarnings([]);
    setSuccessMsg(null);
    try {
      const selected = await openDialog({
        title: 'Select Journal Files',
        filters: [{ name: 'Journal Files', extensions: ['.md', '.txt'] }],
        multiple: true,
      });

      if (!selected) {
        return; // User cancelled
      }

      // Ensure we have an array of file paths
      const filePaths = Array.isArray(selected) ? selected : [selected];
      if (filePaths.length === 0) return;

      setLoading(true);
      let totalEntries = 0;
      const allWarnings: string[] = [];
      let importedCount = 0;

      for (const filePath of filePaths) {
        try {
          const res = await importJournalFile(filePath);
          totalEntries += res.entryCount;
          const fileName = filePath.split(/[\\\/]).pop() || 'File';

          if (res.entryCount === 0) {
            allWarnings.push(`${fileName} imported but no entries found`);
          }
        } catch (e) {
          console.error('Error importing file:', e);
        }
      }

      setSuccessMsg(`Imported ${totalEntries} entries`);
      setWarnings(allWarnings);
    } catch (e) {
      console.error('Error during import:', e);
    }
  };
};

```

```

    }
    if (res.warnings && res.warnings.length > 0) {
      allWarnings.push(...res.warnings.map(w => `[${fileName}] ${w}`));
    }
    importedCount++;
  } catch (err: any) {
    const fileName = filePath.split(/[\\]/).pop() || 'File';
    allWarnings.push(`Failed to import ${fileName}: ${err.message} || err`);
  }
}

if (importedCount > 0) {
  if (totalEntries === 0) {
    setSuccessMsg(null);
  } else {
    setSuccessMsg(`Successfully imported ${totalEntries} entries from
    ${importedCount} file(s)!`);
  }
}

if (allWarnings.length > 0) {
  setWarnings(allWarnings);
}

fetchSources();
} catch (err: any) {
  alert(`Import error: ${err}`);
} finally {
  setLoading(false);
}
};

const handleReimport = async (filePath: string) => {
  setWarnings([]);
  setSuccessMsg(null);
  setLoading(true);
  try {
    const res = await importJournalFile(filePath);
    const fileName = filePath.split(/[\\]/).pop() || 'File';

    if (res.entryCount === 0) {
      setSuccessMsg(null);
      const warnList = [`${fileName} imported but no entries found`];
      if (res.warnings && res.warnings.length > 0) {
        warnList.push(...res.warnings);
      }
      setWarnings(warnList);
    } else {
      setSuccessMsg(`Successfully re-imported ${res.entryCount} entries!`);
      if (res.warnings && res.warnings.length > 0) {
        setWarnings(res.warnings);
      }
    }
  }
}

```



```

    fetchSources();
  } catch (err: any) {
    alert(`Re-import error: ${err}`);
  } finally {
    setLoading(false);
  }
};

const handleRemove = async (filePath: string) => {
  if (!confirm('Are you sure you want to remove this source? All associated
thought-units will be deleted.')) {
    return;
  }
  setWarnings([]);
  setSuccessMsg(null);
  setLoading(true);
  try {
    await removeJournalSource(filePath);
    setSuccessMsg('Source removed successfully.');
```

fetchSources();

```

  } catch (err: any) {
    alert(`Removal error: ${err}`);
  } finally {
    setLoading(false);
  }
};

return (
  <div style={{ padding: 'var(--space-6)', color: 'var(--color-text-primary)' }}>
    /* View Header */
    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center', marginBottom: 'var(--space-6)' }}>
      <div>
        <h1 style={{ fontSize: 'var(--text-xl)', fontWeight: 'var(--weight-bold)'
}}>Journal Sources</h1>
        <p style={{ fontSize: 'var(--text-sm)', color: 'var(--color-text-secondary)',
marginTop: 'var(--space-1)' }}>
          Import your stream-of-consciousness Markdown or Text files.
        </p>
      </div>
      <button
        onClick={handleImport}
        style={{
          background: 'var(--color-primary)',
          color: 'white',
          border: 'none',
          borderRadius: 'var(--radius-md)',
          padding: 'var(--space-2) var(--space-4)',
          fontSize: 'var(--text-sm)',
          fontWeight: 'var(--weight-semibold)',
          cursor: 'pointer',
          display: 'flex',
          alignItems: 'center',

```

```

        gap: 'var(--space-2)',
    }}
        onMouseOver={(e) => (e.currentTarget.style.background =
'var(--color-primary-hover)')}}
        onMouseOut={(e) => (e.currentTarget.style.background =
'var(--color-primary)')}}
    >
        <Plus size={16} />
        Import File
    </button>
</div>

{/* Success Banner */}
{successMsg && (
    <div
        style={{
            background: 'rgba(81, 207, 102, 0.15)',
            border: '1px solid var(--color-success)',
            borderRadius: 'var(--radius-md)',
            padding: 'var(--space-3) var(--space-4)',
            marginBottom: 'var(--space-4)',
            display: 'flex',
            alignItems: 'center',
            gap: 'var(--space-2)',
            color: 'var(--color-success)',
            fontSize: 'var(--text-sm)',
        }}
    >
        <CheckCircle size={16} />
        {successMsg}
    </div>
)}

{/* Warnings Panel */}
{warnings.length > 0 && (
    <div
        style={{
            background: 'rgba(255, 212, 59, 0.12)',
            border: '1px solid var(--color-warning)',
            borderRadius: 'var(--radius-md)',
            padding: 'var(--space-4)',
            marginBottom: 'var(--space-4)',
            color: 'var(--color-warning)',
            fontSize: 'var(--text-sm)',
        }}
    >
        <div style={{ display: 'flex', alignItems: 'center', gap: 'var(--space-2)',
fontWeight: 'var(--weight-bold)', marginBottom: 'var(--space-2)' }}>
            <AlertTriangle size={16} />
            Import Warnings
        </div>
        <ul style={{ paddingLeft: '20px', display: 'flex', flexDirection: 'column',
gap: 'var(--space-1)' }}>

```

```

        {warnings.map((w, idx) => (
            <li key={idx}>{w}</li>
        ))}
    </ul>
</div>
)}

{/* Table grid */}
<div style={{ background: 'var(--color-surface-elevated)', border: '1px solid
var(--color-border)', borderRadius: 'var(--radius-lg)', overflow: 'hidden' }}>
    {loading && sources.length === 0 ? (
        <div style={{ padding: 'var(--space-8)', textAlign: 'center', color:
'var(--color-text-secondary)' }}>
            <RefreshCw size={24} className="spin" style={{ color:
'var(--color-primary)', margin: '0 auto' }} />
        </div>
    ) : sources.length === 0 ? (
        <div style={{ padding: 'var(--space-8)', textAlign: 'center', color:
'var(--color-text-secondary)' }}>
            <FileText size={48} style={{ margin: '0 auto var(--space-4)', opacity: 0.3
}} />
            No journal source files imported yet.
        </div>
    ) : (
        <table style={{ width: '100%', borderCollapse: 'collapse', textAlign: 'left',
fontSize: 'var(--text-sm)' }}>
            <thead>
                <tr style={{ borderBottom: '1px solid var(--color-border)', background:
'var(--color-background)', color: 'var(--color-text-secondary)', fontWeight:
'var(--weight-semibold)' }}>
                    <th style={{ padding: 'var(--space-3) var(--space-4)' }}>File
Details</th>
                    <th style={{ padding: 'var(--space-3) var(--space-4)' }}>Format</th>
                    <th style={{ padding: 'var(--space-3) var(--space-4)' }}>Entries</th>
                    <th style={{ padding: 'var(--space-3) var(--space-4)' }}>Last
Modified</th>
                    <th style={{ padding: 'var(--space-3) var(--space-4)', textAlign:
'right' }}>Actions</th>
                </tr>
            </thead>
            <tbody>
                {sources.map((s) => (
                    <tr key={s.id} style={{ borderBottom: '1px solid var(--color-border)'
}}>
                        <td style={{ padding: 'var(--space-4)' }}>
                            <div style={{ fontWeight: 'var(--weight-semibold)', color:
'var(--color-text-primary)' }}>{s.fileName}</div>
                            <div style={{ fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)', marginTop: '2px', wordBreak: 'break-all'
}}>{s.filePath}</div>
                        </td>
                        <td style={{ padding: 'var(--space-4)', color:
'var(--color-text-primary)' }}>

```

```

        Format {s.formatVersion}
    </td>
        <td style={{ padding: 'var(--space-4)', fontWeight:
'var(--weight-medium)', color: 'var(--color-primary)' }}>
            {s.entryCount}
        </td>
        <td style={{ padding: 'var(--space-4)', color:
'var(--color-text-secondary)', fontSize: 'var(--text-xs)' }}>
            {new Date(s.lastModified).toLocaleDateString()} {new
Date(s.lastModified).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })}
        </td>
        <td style={{ padding: 'var(--space-4)', textAlign: 'right' }}>
            <d
iv style={{ display: 'flex', gap: 'var(--space-2)', justifyContent: 'flex-end' }}>
                <button
                    title="Re-import File"
                    onClick={() => handleReimport(s.filePath)}
                    style={{
                        background: 'none',
                        border: 'none',
                        color: 'var(--color-text-secondary)',
                        cursor: 'pointer',
                        padding: 'var(--space-1)',
                        borderRadius: 'var(--radius-sm)',
                    }}
                    onMouseOver={(e) => (e.currentTarget.style.color =
'var(--color-primary))}
                    onMouseOut={(e) => (e.currentTarget.style.color =
'var(--color-text-secondary))}
                >
                    <RefreshCw size={14} />
                </button>
                <button
                    title="Remove Source"
                    onClick={() => handleRemove(s.filePath)}
                    style={{
                        background: 'none',
                        border: 'none',
                        color: 'var(--color-text-secondary)',
                        cursor: 'pointer',
                        padding: 'var(--space-1)',
                        borderRadius: 'var(--radius-sm)',
                    }}
                    onMouseOver={(e) => (e.currentTarget.style.color =
'var(--color-error))}
                    onMouseOut={(e) => (e.currentTarget.style.color =
'var(--color-text-secondary))}
                >
                    <Trash2 size={14} />
                </button>
            </div>
        </td>
    </tr>

```

```

        ))}
      </tbody>
    </table>
  )}
</div>

{/* Info card */}
<div
  style={{
    background: 'var(--color-surface-elevated)',
    border: '1px solid var(--color-border)',
    borderRadius: 'var(--radius-md)',
    padding: 'var(--space-4)',
    marginTop: 'var(--space-6)',
    display: 'flex',
    gap: 'var(--space-3)',
    alignItems: 'flex-start',
  }}
>
  <Info size={16} style={{ color: 'var(--color-primary)', marginTop: '2px',
flexShrink: 0 }} />
  <div style={{ fontSize: 'var(--text-sm)', color: 'var(--color-text-secondary)',
lineHeight: 'var(--leading-sm)' }}>
    <strong style={{ color: 'var(--color-text-primary)' }}>Formats Auto-Detection
Guide:</strong>
    <ul style={{ marginTop: 'var(--space-2)', display: 'flex', flexDirection:
'column', gap: '4px', paddingLeft: '16px' }}>
      <li>Format 3: Contains <code>#presence</code> or <code>#reminiscence</code>
category tags.</li>
      <li>Format 1: Bullet-point journal style (bullet markers like <code>-</code>
or <code>*</code>).</li>
      <li>Format 2: Plain paragraphs separated by empty lines under date
headers.</li>
    </ul>
  </div>
</div>
</div>
);
};

```

--- END_FILE ---

--- START_FILE: src/components/LetterEditor.tsx ---

```

import React, { useEffect, useState, useRef } from 'react';
import { useCreateBlockNote } from '@blocknote/react';
import { BlockNoteView } from '@blocknote/mantine';
import { schema } from '../blocks/schema';
import { loadLetterContent, saveLetterContent } from '../services/letterService';
import { useAppStore } from '../stores/appStore';
import "@blocknote/mantine/style.css";

```

```

// eslint-disable-next-line @typescript-eslint/no-explicit-any
function normalizeLegacyBlocks(blocks: any[]): any[] {
  if (!Array.isArray(blocks)) return blocks;
  return blocks.map((b) => {
    if (!b) return b;
    if (b.type === "routedEntry") {
      const legacyText = b.props?.entryContent || "";
      const contentArray = Array.isArray(b.content) ? b.content : [];
      const content = contentArray.length > 0 ? contentArray : (legacyText ? [{ type:
"text", text: legacyText, styles: {} }] : []);
      return {
        ...b,
        props: {
          sourceEntryId: b.props?.sourceEntryId || b.props?.entrySourceFile || "",
          sourceDate: b.props?.sourceDate || b.props?.entryDate || "",
          category: b.props?.category || b.props?.entryCategory || "uncategorized",
        },
        content,
      };
    }
    if (b.type === "scaffoldBlock") {
      const legacyTitle = b.props?.sectionLabel || b.props?.title || "Scaffold Section";
      const legacyTimeframe = b.props?.scaffoldId || b.props?.timeframe || "";

      const contentArray = Array.isArray(b.content) ? b.content : [];
      const nestedBlocks = contentArray.filter((x: any) => x && typeof x === 'object' &&
x.type);
      const remainingInline = contentArray.filter((x: any) => !x || typeof x !==
'object' || !x.type);

      const legacyChildren = Array.isArray(b.children) ? b.children : [];
      const mergedChildren = [...legacyChildren, ...nestedBlocks];

      return {
        ...b,
        props: {
          scaffoldId: legacyTimeframe,
          sectionLabel: legacyTitle,
          sourceEntryId: b.props?.sourceEntryId || "",
        },
        content: remainingInline,
        children: normalizeLegacyBlocks(mergedChildren),
      };
    }
    if (Array.isArray(b.children)) {
      return { ...b, children: normalizeLegacyBlocks(b.children) };
    }
    return b;
  });
}

interface LetterEditorProps {

```

```

letterId: string;
onBlocksChange: (blocks: any[]) => void;
onSaveStateChange: (saving: boolean, message: string) => void;
}

export const LetterEditor: React.FC<LetterEditorProps> = ({
  letterId,
  onBlocksChange,
  onSaveStateChange,
}) => {
  const [initialBlocks, setInitialBlocks] = useState<any[] | null>(null);

  // 1. Fetch initial content on mount
  useEffect(() => {
    const loadContent = async () => {
      try {
        const res = await loadLetterContent(letterId);
        let blocks: any[] = [];
        if (res.blocksJson) {
          try {
            const parsed = JSON.parse(res.blocksJson);
            if (Array.isArray(parsed)) {
              blocks = parsed;
            } else {
              // Convert plain text to paragraph blocks
              blocks = res.blocksJson.split('\n\n').map((p) => ({
                type: 'paragraph',
                content: [{ type: 'text', text: p, styles: {} }],
              }));
            }
          } catch {
            blocks = res.blocksJson.split('\n\n').map((p) => ({
              type: 'paragraph',
              content: [{ type: 'text', text: p, styles: {} }],
            }));
          }
        }
        if (blocks.length === 0) {
          blocks = [{ type: 'paragraph', content: [] }];
        }
        const normalized = normalizeLegacyBlocks(blocks); // migrate old block shapes
        setInitialBlocks(normalized);
        onBlocksChange(normalized);
      } catch (err) {
        console.error('Failed to load letter content:', err);
        setInitialBlocks([{ type: 'paragraph', content: [] }]);
      }
    };

    loadContent();
  }, [letterId]);

  if (initialBlocks === null) {

```

```

return (
  <div
    style={{
      display: 'flex',
      justifyContent: 'center',
      alignItems: 'center',
      height: '150px',
      color: 'var(--color-text-secondary)',
    }}
  >
    Loading letter content...
  </div>
);
}

return (
  <ActualBlockEditor
    letterId={letterId}
    initialBlocks={initialBlocks}
    onBlocksChange={onBlocksChange}
    onSaveStateChange={onSaveStateChange}
  />
);
};

interface ActualBlockEditorProps {
  letterId: string;
  initialBlocks: any[];
  onBlocksChange: (blocks: any[]) => void;
  onSaveStateChange: (saving: boolean, message: string) => void;
}

const ActualBlockEditor: React.FC<ActualBlockEditorProps> = ({
  letterId,
  initialBlocks,
  onBlocksChange,
  onSaveStateChange,
}) => {
  const [isDarkMode, setIsDarkMode] = useState(() => {
    if (typeof window !== 'undefined') {
      return window.matchMedia('(prefers-color-scheme: dark)').matches;
    }
    return true;
  });

  useEffect(() => {
    const mediaQuery = window.matchMedia('(prefers-color-scheme: dark)');
    const handler = (e: MediaQueryListEvent) => {
      setIsDarkMode(e.matches);
    };
    if (mediaQuery.addEventListener) {
      mediaQuery.addEventListener('change', handler);
      return () => mediaQuery.removeEventListener('change', handler);
    }
  });

```



```

    } else {
      mediaQuery.addListener(handler);
      return () => mediaQuery.removeListener(handler);
    }
  }, []));

const editor = useCreateBlockNote({
  schema,
  initialContent: initialBlocks,
});

const { routingQueue, removeFromRoutingQueue } = useAppStore();
const saveTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null);

// 2. Consume routing queue items for this specific active letter tab
useEffect(() => {
  const myQueueItems = routingQueue.filter((item) => item.targetLetterId ===
letterId);
  if (myQueueItems.length === 0) return;

  editor.transact(() => {
    myQueueItems.forEach((item) => {
      const blockToInsert: any = {
        type: item.blockType || "routedEntry",
        props: item.blockType === "scaffoldBlock" ? {
          scaffoldId: item.scaffoldId || "",
          sectionLabel: item.sectionLabel || "Scaffold Section",
          sourceEntryId: item.sourceThoughtUnit.id,
        } : {
          sourceEntryId: item.sourceThoughtUnit.id,
          sourceDate: item.sourceThoughtUnit.date ?? "",
          category: item.sourceThoughtUnit.category,
        },
        content: item.blockType === "scaffoldBlock" ? [] : [{
          type: "text",
          text: item.sourceThoughtUnit.content,
          styles: {},
        }],
        ...(item.blockType === "scaffoldBlock" ? {
          children: [{
            type: "routedEntry",
            props: {
              sourceEntryId: item.sourceThoughtUnit.id,
              sourceDate: item.sourceThoughtUnit.date ?? "",
              category: item.sourceThoughtUnit.category,
            },
            content: [{
              type: "text",
              text: item.sourceThoughtUnit.content,
              styles: {},
            }]
          }
        ] : {}),
      };
    });
  });
}, [letterId]);

```

```

    };

    const lastBlock = editor.document[editor.document.length - 1];
    if (lastBlock) {
        editor.insertBlocks([blockToInsert], lastBlock, "after");
    } else {
        const firstBlock = editor.document[0];
        if (firstBlock) {
            editor.insertBlocks([blockToInsert], firstBlock, "before");
        }
    }
    });
});

// Save immediately since content is routed in
const updatedContent = JSON.stringify(editor.document);
saveLetterContent(letterId, updatedContent).catch(console.error);
onBlocksChange(editor.document);

// Remove from store routing queue
myQueueItems.forEach((item) => {
    removeFromRoutingQueue(item.id);
});
}, [routingQueue, letterId, editor]);

// 3. Debounced Auto-save (3 seconds) + onChange notify
const handleEditorChange = () => {
    onBlocksChange(editor.document);
    onSaveStateChange(true, 'Typing...');

    if (saveTimeoutRef.current) {
        clearTimeout(saveTimeoutRef.current);
    }

    saveTimeoutRef.current = setTimeout(async () => {
        try {
            const content = JSON.stringify(editor.document);
            await saveLetterContent(letterId, content);
            onSaveStateChange(false, `Saved at ${new Date().toLocaleTimeString()}`);
        } catch (err) {
            console.error('Auto-save failed:', err);
            onSaveStateChange(false, 'Save failed – retrying...');
        }
    }, 3000);
};

// 3.5. Register standard editor change listener
useEffect(() => {
    if (!editor) return;
    const unsub = editor.onChange(() => {
        handleEditorChange();
    });
    return () => {

```

```

    unsub?();
  };
}, [editor]);

// 4. Flush-on-deactivate / FLUSH-BEFORE-UNMOUNT cleanup
useEffect(() => {
  return () => {
    if (saveTimeoutRef.current) {
      clearTimeout(saveTimeoutRef.current);
    }
    // Immediate database write during unmount to ensure 0 data loss
    const finalContent = JSON.stringify(editor.document);
    saveLetterContent(letterId, finalContent).catch(console.error);
  };
}, [letterId, editor]);

// 5. Ctrl+S / Cmd+S manual force-save handler
useEffect(() => {
  const handleKeyDown = async (e: KeyboardEvent) => {
    if ((e.ctrlKey || e.metaKey) && e.key.toLowerCase() === 's') {
      e.preventDefault();
      if (saveTimeoutRef.current) {
        clearTimeout(saveTimeoutRef.current);
      }
      onSaveStateChange(true, 'Saving...');
      try {
        const content = JSON.stringify(editor.document);
        await saveLetterContent(letterId, content);
        onSaveStateChange(false, `Saved at ${new Date().toLocaleTimeString()}`);
      } catch (err) {
        console.error('Manual save failed:', err);
      }
    }
  };

  onSaveStateChange(false, 'Save failed – retrying...');
}, [letterId, editor, onSaveStateChange]);

window.addEventListener('keydown', handleKeyDown);
return () => {
  window.removeEventListener('keydown', handleKeyDown);
};
}, [letterId, editor, onSaveStateChange]);

const onDrop = (e: React.DragEvent) => {
  const raw = e.dataTransfer.getData('application/x-thought-unit');
  if (!raw || !editor) return;
  e.preventDefault();
  try {
    const u = JSON.parse(raw);
    const blockToInsert: any = {
      type: "routedEntry",
      props: {
        sourceEntryId: u.id,

```

```

        sourceDate:    u.date ?? "",
        category:      u.category,
    },
    content: [{
        type: "text",
        text: u.content,
        styles: {},
    }]
};

const lastBlock = editor.document[editor.document.length - 1];
if (lastBlock) {
    editor.insertBlocks([blockToInsert], lastBlock, "after");
} else {
    const firstBlock = editor.document[0];
    if (firstBlock) {
        editor.insertBlocks([blockToInsert], firstBlock, "before");
    }
}
} catch (err) {
    console.error("Drop parse failed:", err);
}
};

const onDragOver = (e: React.DragEvent) => {
    if (e.dataTransfer.types.includes('application/x-thought-unit')) {
        e.preventDefault();
        e.dataTransfer.dropEffect = 'copy';
    }
};

return (
    <div
        onDrop={onDrop}
        onDragOver={onDragOver}
        style={{
            display: 'flex',
            flexDirection: 'column',
            flex: 1,
            width: '100%',
            height: '100%',
            minHeight: 0,
            background: 'var(--color-surface)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-lg)',
            color: 'var(--color-text-primary)',
            fontSize: 'var(--text-base)',
        }}
    >
        <div style={{ flex: 1, minHeight: 0, overflow: 'auto', padding: 'var(--space-4)'
        >
            <BlockNoteView
                editor={editor}

```

```

        theme={isDarkMode ? "dark" : "light"}
      />
    </div>
  </div>
);
};

```

--- END_FILE ---

--- START_FILE: src/components/PenpalPanel.tsx ---

```

import React, { useState, useEffect } from 'react';
import { useAppStore } from '../stores/appStore';
import { createPenpal, updatePenpal, addCorrespondence, getCorrespondence } from
'../services/penpalService';
import { Penpal, Correspondence } from '../types';
import { UserPlus, Edit, Mail, Send, Calendar, Globe, Tag, BookOpen, ChevronLeft, Save }
from 'lucide-react';

export const PenpalPanel: React.FC = () => {
  const { penpals, loadPenpals, openNewLetter } = useAppStore();

  const [selectedPenpalId, setSelectedPenpalId] = useState<string | null>(null);
  const [viewMode, setViewMode] = useState<'list' | 'create' | 'edit' |
'detail'>('list');

  // Form States
  const [name, setName] = useState('');
  const [country, setCountry] = useState('');
  const [interests, setInterests] = useState('');
  const [topics, setTopics] = useState('');
  const [notes, setNotes] = useState('');

  // Correspondence States
  const [correspondenceList, setCorrespondenceList] = useState<Correspondence[]>([]);
  const [showAddCorr, setShowAddCorr] = useState(false);
  const [corrDirection, setCorrDirection] = useState<'sent' | 'received'>('received');
  const [corrContent, setCorrContent] = useState('');
  const [corrDate, setCorrDate] = useState('');

  // Load penpals on mount
  useEffect(() => {
    loadPenpals();
  }, []);

  const selectedPenpal = penpals.find((p) => p.id === selectedPenpalId);

  // Load correspondence whenever a penpal is selected/opened
  useEffect(() => {
    if (selectedPenpalId) {
      fetchCorrespondence(selectedPenpalId);
    }
  });

```

```

    }
  }, [selectedPenpalId]);

const fetchCorrespondence = async (id: string) => {
  try {
    const data = await getCorrespondence(id);
    setCorrespondenceList(data);
  } catch (e) {
    console.error('Failed to fetch correspondence:', e);
  }
};

const handleCreate = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!name.trim()) return alert('Name is required');

  try {
    const newPenpal = await createPenpal(name, country, interests, topics, notes);
    await loadPenpals();
    setSelectedPenpalId(newPenpal.id);
    setViewMode('detail');
    resetForm();
  } catch (err: any) {
    alert(`Error creating penpal: ${err}`);
  }
};

const handleUpdate = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!selectedPenpalId || !name.trim()) return;

  try {
    await updatePenpal(selectedPenpalId, name, country, interests, topics, notes);
    await loadPenpals();
    setViewMode('detail');
    resetForm();
  } catch (err: any) {
    alert(`Error updating penpal: ${err}`);
  }
};

const handleAddCorrespondenceSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!selectedPenpalId || !corrContent.trim() || !corrDate) {
    return alert('Content and Letter Date are required.');
```

```

    } catch (err: any) {
      alert(`Error logging letter: ${err}`);
    }
  };

const startEdit = (penpal: Penpal) => {
  setName(penpal.name);
  setCountry(penpal.country);
  setInterests(penpal.interests);
  setTopics(penpal.topics);
  setNotes(penpal.notes);
  setViewMode('edit');
};

const resetForm = () => {
  setName('');
  setCountry('');
  setInterests('');
  setTopics('');
  setNotes('');
};

return (
  <div style={{ display: 'flex', flexDirection: 'column', height: '100%' }}>
    {/* Panel Header */}
    <div style={{ padding: 'var(--space-4)', borderBottom: '1px solid
var(--color-border)', background: 'var(--color-surface)' }}>
      <h2 style={{ fontSize: 'var(--text-lg)', fontWeight: 'var(--weight-semibold)',
color: 'var(--color-text-primary)' }}>Penpal Workspace</h2>
    </div>

    {/* Main Workspace Scroll Area */}
    <div style={{ flex: 1, overflowY: 'auto', padding: 'var(--space-4)' }}>
      {viewMode === 'list' && (
        <div style={{ display: 'flex', flexDirection: 'column', gap: 'var(--space-4)'
}}}>

        <button
          onClick={() => {
            resetForm();
            setViewMode('create');
          }}
          style={{
            width: '100%',
            background: 'var(--color-primary)',
            color: 'white',
            border: 'none',
            borderRadius: 'var(--radius-md)',
            padding: 'var(--space-2) var(--space-3)',
            fontSize: 'var(--text-sm)',
            fontWeight: 'var(--weight-semibold)',
            cursor: 'pointer',
            display: 'flex',
            alignItems: 'center',

```

```

        justifyContent: 'center',
        gap: 'var(--space-2)',
    }}
    onMouseOver={(e) => (e.currentTarget.style.backgroundColor =
'var(--color-primary-hover)')}
    onMouseOut={(e) => (e.currentTarget.style.backgroundColor =
'var(--color-primary)')}
    >
    <UserPlus size={16} />
    Add New Penpal
</button>

    { /* List View */ }
    <div style={{ display: 'flex', flexDirection: 'column', gap:
'var(--space-2)' }}>
        {penpals.length === 0 ? (
            <div style={{ textAlign: 'center', color: 'var(--color-text-secondary)',
padding: 'var(--space-8)' }}>
                No penpals added yet.
            </div>
        ) : (
            penpals.map((p) => (
                <div
                    key={p.id}
                    onClick={() => {
                        setSelectedPenpalId(p.id);
                        setViewMode('detail');
                    }}
                    style={{
                        background: 'var(--color-surface-elevated)',
                        border: '1px solid var(--color-border)',
                        borderRadius: 'var(--radius-md)',
                        padding: 'var(--space-3)',
                        cursor: 'pointer',
                        transition: 'border var(--transition-fast)',
                        display: 'flex',
                        flexDirection: 'column',
                        gap: '4px',
                    }}
                    onMouseOver={(e) => (e.currentTarget.style.borderColor =
'var(--color-primary)')}
                    onMouseOut={(e) => (e.currentTarget.style.borderColor =
'var(--color-border)')}
                >
                    <div style={{ display: 'flex', justifyContent: 'space-between',
alignItems: 'flex-start' }}>
                        <div style={{ fontWeight: 'var(--weight-semibold)', color:
'var(--color-text-primary)' }}>{p.name}</div>
                        {p.letterCount !== undefined && p.letterCount > 0 && (
                            <span style={{ fontSize: '10px', background: 'rgba(2, 132, 199,
0.1)', color: 'var(--color-primary)', borderRadius: 'var(--radius-sm)', padding: '1px
6px', fontWeight: 'var(--weight-semibold)' }}>
                                {p.letterCount} letter{p.letterCount > 1 ? 's' : ''}
                            </span>
                        )}
                    </div>
                </div>
            )
        )}
    </div>

```



```

        </span>
      )}
    </div>
    <div style={{ display: 'flex', justifyContent: 'space-between',
alignItems: 'center', fontSize: 'var(--text-xs)', color: 'var(--color-text-secondary)'
}}>
      <div style={{ display: 'flex', alignItems: 'center', gap: '4px'
}}>
        <Globe size={10} />
        {p.country || 'No Country'}
      </div>
      {p.lastLetterDate && (
        <div style={{ display: 'flex', alignItems: 'center', gap: '2px',
fontSize: '10px' }}>
          <Calendar size={10} />
          {p.lastLetterDate}
        </div>
      )}
    </div>
  </div>
))
)}
</div>
</div>
)}

{/* Create Profile Form */}
{(viewModel === 'create' || viewModel === 'edit') && (
  <form onSubmit={viewModel === 'create' ? handleCreate : handleUpdate} style={{
display: 'flex', flexDirection: 'column', gap: 'var(--space-3)' }}>
    <div style={{ display: 'flex', alignItems: 'center', gap: 'var(--space-2)',
marginBottom: 'var(--space-2)' }}>
      <button
        type="button"
        onClick={() => setViewModel(viewModel === 'create' ? 'list' : 'detail')}
        style={{ background: 'none', border: 'none', color:
'var(--color-text-secondary)', cursor: 'pointer', display: 'flex', alignItems: 'center',
padding: 0 }}
      >
        <ChevronLeft size={16} />
      </button>
      <h3 style={{ fontSize: 'var(--text-base)', fontWeight:
'var(--weight-semibold)' }}>
        {viewModel === 'create' ? 'Create Penpal Profile' : 'Edit Penpal
Profile'}
      </h3>
    </div>

    <div>
      <label style={{ display: 'block', fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)', marginBottom: '4px', fontWeight: 'var(--weight-semibold)'
}}>NAME *</label>
      <input

```

```

        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Penpal's Name"
        required
        style={{
            width: '100%',
            background: 'var(--color-surface-elevated)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-md)',
            padding: 'var(--space-2) var(--space-3)',
            color: 'var(--color-text-primary)',
            fontSize: 'var(--text-sm)',
        }}
    />
</div>

<div>
    <label style={{ display: '
block', fontSize: 'var(--text-xs)', color: 'var(--color-text-secondary)', marginBottom:
'4px', fontWeight: 'var(--weight-semibold)' }}>COUNTRY</label>
    <input
        type="text"
        value={country}
        onChange={(e) => setCountry(e.target.value)}
        placeholder="Country Location"
        style={{
            width: '100%',
            background: 'var(--color-surface-elevated)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-md)',
            padding: 'var(--space-2) var(--space-3)',
            color: 'var(--color-text-primary)',
            fontSize: 'var(--text-sm)',
        }}
    />
</div>

<div>
    <label style={{ display: 'block', fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)', marginBottom: '4px', fontWeight: 'var(--weight-semibold)'
}}>INTERESTS</label>
    <input
        type="text"
        value={interests}
        onChange={(e) => setInterests(e.target.value)}
        placeholder="e.g. Reading, Hiking, Cooking"
        style={{
            width: '100%',
            background: 'var(--color-surface-elevated)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-md)',
            padding: 'var(--space-2) var(--space-3)',

```

```

        color: 'var(--color-text-primary)',
        fontSize: 'var(--text-sm)',
      }}
    />
  </div>

  <div>
    <label style={{ display: 'block', fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)', marginBottom: '4px', fontWeight: 'var(--weight-semibold)'
}}>TOPICS</label>
    <input
      type="text"
      value={topics}
      onChange={(e) => setTopics(e.target.value)}
      placeholder="Topics to write about..."
      style={{
        width: '100%',
        background: 'var(--color-surface-elevated)',
        border: '1px solid var(--color-border)',
        borderRadius: 'var(--radius-md)',
        padding: 'var(--space-2) var(--space-3)',
        color: 'var(--color-text-primary)',
        fontSize: 'var(--text-sm)',
      }}
    />
  </div>

  <div>
    <label style={{ display: 'block', fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)', marginBottom: '4px', fontWeight: 'var(--weight-semibold)'
}}>NOTES</label>
    <textarea
      value={notes}
      onChange={(e) => setNotes(e.target.value)}
      placeholder="Personal notes, biography details, past discussions..."
      rows={4}
      style={{
        width: '100%',
        background: 'var(--color-surface-elevated)',
        border: '1px solid var(--color-border)',
        borderRadius: 'var(--radius-md)',
        padding: 'var(--space-2) var(--space-3)',
        color: 'var(--color-text-primary)',
        fontSize: 'var(--text-sm)',
        resize: 'vertical',
        fontFamily: 'inherit',
      }}
    />
  </div>

  <button
    type="submit"
    style={{

```

```

        width: '100%',
        background: 'var(--color-primary)',
        color: 'white',
        border: 'none',
        borderRadius: 'var(--radius-md)',
        padding: 'var(--space-2)',
        fontSize: 'var(--text-sm)',
        fontWeight: 'var(--weight-semibold)',
        cursor: 'pointer',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'center',
        gap: 'var(--space-2)',
        marginTop: 'var(--space-2)',
    }}
    >
    <Save size={16} />
    {viewModel === 'create' ? 'Save Profile' : 'Update Profile'}
    </button>
</form>
)}

{/* Profile Detail View */}
{viewModel === 'detail' && selectedPenpal && (
    <div style={{ display: 'flex', flexDirection: 'column', gap: 'var(--space-4)'
}}}>

    {/* Navigation back and edit buttons */}
    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center' }}>

        <button
            onClick={() => setViewModel('list')}
            style={{
                background: 'none',
                border: 'none',
                color: 'var(--color-text-secondary)',
                cursor: 'pointer',
                display: 'flex',
                alignItems: 'center',
                gap: '4px',
                fontSize: 'var(--text-xs)',
                padding: 0,
            }}
        >
            <ChevronLeft size={14} />
            Penpals List
        </button>

        <button
            onClick={() => startEdit(selectedPenpal)}
            style={{
                background: 'var(--color-surface-elevated)',
                border: '1px solid var(--color-border)',
                borderRadius: 'var(--radius-sm)',

```

```

        color: 'var(--color-text-primary)',
        cursor: 'pointer',
        fontSize: 'var(--text-xs)',
        padding: 'var(--space-1) var(--space-2)',
        display: 'flex',
        alignItems: 'center',
        gap: '4px',
    }}
    >
    <Edit size={10} />
    Edit Profile
</button>
</div>

{/* Profile Summary Card */}
<div style={{ background: 'var(--color-surface-elevated)', border: '1px
solid var(--color-border)', borderRadius: 'var(--radius-lg)', padding: 'var(--space-4)',
display: 'flex', flexDirection: 'column', gap: 'var(--space-3)' }}>
    <div>
        <h3 style={{ fontSize: 'var(--text-lg)', fontWeight:
'var(--weight-bold)', color: 'var(--color-text-primary)' }}>{selectedPenpal.name}</h3>
        <div style={{ fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)', marginTop: '6px', display: 'flex', flexWrap: 'wrap',
alignItems: 'center', gap: '12px' }}>
            <div style={{ display: 'flex', alignItems: 'center', gap: '4px' }}>
                <Globe size={12} />
                {selectedPenpal.country || 'No Country Registered'}
            </div>
            <div style={{ display: 'flex', alignItems: 'center', gap: '4px' }}>
                <Mail size={12} />
                {selectedPenpal.letterCount || 0} letter{(selectedPenpal.letterCount
|| 0) !== 1 ? 's' : ''}
            </div>
            {selectedPenpal.lastLetterDate && (
                <div style={{ display: 'flex', alignItems: 'center', gap: '4px' }}>
                    <Calendar size={12} />
                    Last: {selectedPenpal.lastLetterDate}
                </div>
            )}
        </div>
    </div>
</div>

{/* Action Buttons */}
<button
    onClick={async () => {
        try {
            await openNewLetter(selectedPenpal.id);
        } catch (e: any) {
            alert(`Failed to open letter draft: ${e.message || e}`);
        }
    }}
    style={{
        width: '100%',

```

```

        background: 'var(--color-primary)',
        color: 'white',
        border: 'none',
        borderRadius: 'var(--radius-md)',
        padding: 'var(--space-2)',
        fontSize: 'var(--text-sm)',
        fontWeight: 'var(--weight-semibold)',
        cursor: 'pointer',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'center',
        gap: 'var(--space-2)',
    }}
    onMouseOver={(e) => (e.currentTarget.style.background =
'var(--color-primary-hover)')}
    onMouseOut={(e) => (e.currentTarget.style.background =
'var(--color-primary)')}
    >
    <Mail size={16} />
    Write Letter Draft
</button>
</div>

{/* Structured details */}
    <div style={{ display: 'flex', flexDirection: 'column', gap:
'var(--space-3)' }}>
    <div>
        <div style={{ fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)', fontWeight: 'var(--weight-bold)', textTransform:
'uppercase', marginBottom: '4px', display: 'flex', alignItems: 'center', gap: '4px' }}>
            <Tag size={10} />
            Interests
        </div>
        <div style={{ fontSize: 'var(--text-sm)', color:
'var(--color-text-primary)', paddingLeft: '14px' }}>
            {selectedPenpal.interests || 'None specified'}
        </div>
    </div>

    <div>
        <div style={{ fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)', fontWeight: 'var(--weight-bold)', textTransform:
'uppercase', marginBottom: '4px', display: 'flex', alignItems: 'center', gap: '4px' }}>
            <BookOpen size={10} />
            Discussable Topics
        </div>
        <div style={{ fontSize: 'var(--text-sm)', color:
'var(--color-text-primary)', paddingLeft: '14px' }}>
            {selectedPenpal.topics || 'None specified'}
        </div>
    </div>

    <div>

```

```

        <div style={{ fontSize: 'v
ar(--text-xs)', color: 'var(--color-text-secondary)', fontWeight: 'var(--weight-bold)',
textTransform: 'uppercase', marginBottom: '4px', display: 'flex', alignItems: 'center',
gap: '4px' }}>
            <Globe size={10} />
            Notes & Bio
        </div>

        <div style={{ fontSize: 'var(--text-sm)', color:
'var(--color-text-primary)', paddingLeft: '14px', whiteSpace: 'pre-wrap' }}>
            {selectedPenpal.notes || 'No biographical notes recorded.'}
        </div>
    </div>
</div>

    {/* Correspondence Section */}
    <div style={{ borderTop: '1px solid var(--color-border)', paddingTop:
'var(--space-4)', marginTop: 'var(--space-2)' }}>
        <div style={{ display: 'flex', justifyContent: 'space-between',
alignItems: 'center', marginBottom: 'var(--space-3)' }}>
            <h4 style={{ fontSize: 'var(--text-sm)', fontWeight:
'var(--weight-semibold)', color: 'var(--color-text-primary)' }}>Letters Archive</h4>
            <button
                onClick={() => setShowAddCorr(!showAddCorr)}
                style={{
                    background: 'none',
                    border: 'none',
                    color: 'var(--color-primary)',
                    cursor: 'pointer',
                    fontSize: 'var(--text-xs)',
                    fontWeight: 'var(--weight-semibold)',
                    padding: 0,
                }}
            >
                {showAddCorr ? 'Cancel' : 'Log Past Letter'}
            </button>
        </div>

        {/* Log Correspondence Form */}
        {showAddCorr && (
            <form onSubmit={handleAddCorrespondenceSubmit} style={{ background:
'var(--color-surface-elevated)', border: '1px solid var(--color-border)', borderRadius:
'var(--radius-md)', padding: 'var(--space-3)', display: 'flex', flexDirection: 'column',
gap: 'var(--space-2)', marginBottom: 'var(--space-3)' }}>
                <div style={{ display: 'flex', gap: 'var(--space-2)' }}>
                    <button
                        type="button"
                        onClick={() => setCorrDirection('received')}
                        style={{
                            flex: 1,
                            background: corrDirection === 'received' ?
'var(--color-primary)' : 'var(--color-surface)',
                            color: corrDirection === 'received' ? 'white' :
'var(--color-text-secondary)',

```

```

        border: '1px solid var(--color-border)',
        borderRadius: 'var(--radius-sm)',
        padding: '4px 0',
        fontSize: 'var(--text-xs)',
        fontWeight: 'var(--weight-semibold)',
        cursor: 'pointer',
      }}
    >
      Received
    </button>
    <button
      type="button"
      onClick={() => setCorrDirection('sent')}
      style={{
        flex: 1,
        background: corrDirection === 'sent' ? 'var(--color-primary)' :
'var(--color-surface)',
        color: corrDirection === 'sent' ? 'white' :
'var(--color-text-secondary)',
        border: '1px solid var(--color-border)',
        borderRadius: 'var(--radius-sm)',
        padding: '4px 0',
        fontSize: 'var(--text-xs)',
        fontWeight: 'var(--weight-semibold)',
        cursor: 'pointer',
      }}
    >
      Sent
    </button>
  </div>

<div>
  <input
    type="date"
    value={corrDate}
    onChange={(e) => setCorrDate(e.target.value)}
    required
    style={{
      width: '100%',
      background: 'var(--color-surface)',
      border: '1px solid var(--color-border)',
      borderRadius: 'var(--radius-sm)',
      padding: '4px 8px',
      color: 'var(--color-text-primary)',
      fontSize: 'var(--text-xs)',
    }}
  />
</div>

<div>
  <textarea
    value={corrContent}
    onChange={(e) => setCorrContent(e.target.value)}
  >

```



```

placeholder="Letter content (plain text)"
rows={3}
required
style={{
  width: '100%',
  background: 'var(--color-surface)',
  border: '1px solid var(--color-border)',
  borderRadius: 'var(--radius-sm)',
  padding: '4px 8px',
  color: 'var(--color-text-primary)',
  fontSize: 'var(--text-xs)',
  resize: 'vertical',
}}
/>
</div>

```

```

<button
  type="submit"
  style={{
    background: 'var(--color-primary)',
    color: 'white',
    border: 'none',
    borderRadius: 'var(--radius-sm)',
    padding: 'var(--space-1) 0',
    fontSize: 'var(--text-xs)',
    fontWeight: 'var(--weight-semibold)',
    cursor: 'pointer',
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'center',
    gap: '4px',
  }}
>
  <Send size={10} />
  Log Record
</button>
</form>
)}

```

```

{/* Correspondence history list */}
    <div style={{ display: 'flex', flexDirection: 'column', gap:
'var(--space-2)' }}>
      {correspondenceList.length === 0 ? (
        <div style={{ fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)', fontStyle: 'italic', textAlign: 'center', padding:
'var(--space-4)' }}>
          No recorded correspondence.
        </div>
      ) : (
        correspondenceList.map((c) => (
          <div key={c.id} style={{ background:
'var(--color-surface-elevated)', border: '1px solid var(--color-border)', borderRadius:
'var(--radius-md)', padding: 'var(--space-3)' }}>

```

```

        <div style={{ display: 'flex', justifyContent: 'space-between',
alignItems: 'center', marginBottom: 'var(--space-1)' }}>
            <span
                style={{
                    fontSize: 'var(--text-xs)',
                    fontWeight: 'var(--weight-bold)',
                    color: c.direction === 'sent' ? 'var(--color-primary)' :
'var(--color-success)',
                }}
            >
                {c.direction === 'sent' ? 'Sent Letter' : 'Received Letter'}
            </span>
            <span style={{ fontSize: '10px', color:
'var(--color-text-secondary)', display: 'flex', alignItems: 'center', gap: '2px' }}>
                <Calendar size={10} />
                {c.letterDate}
            </span>
        </div>
        <div style={{ fontSize: 'var(--text-sm)', color:
'var(--color-text-primary)', whiteSpace: 'pre-wrap', lineHeight: 'var(--leading-sm)' }}>
            {c.content}
        </div>
    </div>
))
)}
</div>
</div>
</div>
)}
</div>
</div>
);
};

```

--- END_FILE ---

--- START_FILE: src/components/ScaffoldModal.tsx ---

```

import React, { useState, useMemo, useRef } from 'react';
import { useAppStore } from '../stores/appStore';
import { getThoughtUnits } from '../services/journalService';
import { loadLetterContent } from '../services/letterService';
import { ThoughtUnit } from '../types';
import { X, Calendar, Sparkles, Check } from 'lucide-react';
import { useVirtualizer } from '@tanstack/react-virtual';

interface ScaffoldModalProps {
    onClose: () => void;
}

export const ScaffoldModal: React.FC<ScaffoldModalProps> = ({ onClose }) => {

```

```

const { penpals, openLetters, openNewLetter, addToRoutingQueue } = useAppStore();

const [startDate, setStartDate] = useState<string>('');
const [endDate, setEndDate] = useState<string>('');
const [loading, setLoading] = useState<boolean>(false);
const [entries, setEntries] = useState<ThoughtUnit[]>([]);
const [selectedEntryIds, setSelectedEntryIds] = useState<Set<string>>(new Set());
const [selectedPenpalIds, setSelectedPenpalIds] = useState<Set<string>>(new Set());
const [fetched, setFetched] = useState<boolean>(false);

// Timeframe Presets
const handlePreset = (preset: 'week' | 'month') => {
  const today = new Date();
  const pad = (n: number) => n.toString().padStart(2, '0');
  const formatDate = (d: Date) => `${d.getFullYear()}-${pad(d.getMonth() +
1)}-${pad(d.getDate())}`;

  if (preset === 'week') {
    const start = new Date(today);
    const day = start.getDay();
    const diff = start.getDate() - day + (day === 0 ? -6 : 1);
    const monday = new Date(start.setDate(diff));
    setStartDate(formatDate(monday));
    setEndDate(formatDate(today));
  } else if (preset === 'month') {
    const firstDay = new Date(today.getFullYear(), today.getMonth(), 1);
    setStartDate(formatDate(firstDay));
    setEndDate(formatDate(today));
  }
};

// Fetch entries
const handleFetch = async () => {
  if (!startDate || !endDate) {
    alert('Please select both start and end dates.');
```

return;

```

  }
  setLoading(true);
  setFetched(true);
  try {
    const data = await getThoughtUnits(startDate, endDate, null);
    setEntries(data);
    // Auto-select all fetched entries
    setSelectedEntryIds(new Set(data.map((tu) => tu.id)));
  } catch (e) {
    console.error('Failed to fetch entries for scaffold:', e);
    alert('Failed to load journal entries.');
```

} finally {

```

    setLoading(false);
  }
};

// Group entries by date
```

```

const groupedEntries = useMemo(() => {
  const groups: Record<string, ThoughtUnit[]> = {};
  entries.forEach((tu) => {
    const date = tu.date || 'Undated';
    if (!groups[date]) groups[date] = [];
    groups[date].push(tu);
  });
  // Sort dates in ascending order
  return Object.keys(groups)
    .sort((a, b) => a.localeCompare(b))
    .map((date) => ({
      date,
      items: groups[date] || [],
    }));
}, [entries]);

```

```

const parentRef = useRef<HTMLDivElement>(null);

```

```

const flatList = useMemo(() => {
  const list: Array<{
    type: 'header' | 'item';
    id: string;
    date?: string;
    item?: ThoughtUnit;
    items?: ThoughtUnit[];
  }> = [];

  groupedEntries.forEach((group) => {
    list.push({
      type: 'header',
      id: `header-${group.date}`,
      date: group.date,
      items: group.items,
    });
    group.items.forEach((item) => {
      list.push({
        type: 'item',
        id: item.id,
        item,
      });
    });
  });

```

```

  return list;
}, [groupedEntries]);

```

```

const rowVirtualizer = useVirtualizer({
  count: flatList.length,
  getScrollElement: () => parentRef.current,
  estimateSize: (index) => {
    const item = flatList[index];
    if (!item) return 0;
    if (item.type === 'header') return 36;

```

```

    const contentLength = item.item?.content?.length || 0;
    if (contentLength > 150) return 120;
    if (contentLength > 80) return 96;
    return 78;
  },
  overscan: 10,
});

```

// Toggle entry selection

```

const handleToggleEntry = (id: string) => {
  const next = new Set(selectedEntryIds);
  if (next.has(id)) {
    next.delete(id);
  } else {
    next.add(id);
  }
  setSelectedEntryIds(next);
};

```

// Toggle all entries for a date

```

const handleToggleDate = (items: ThoughtUnit[]) => {
  const next = new Set(selectedEntryIds);
  const itemIds = items.map((i) => i.id);
  const allSelected = itemIds.every((id) => next.has(id));

  if (allSelected) {
    itemIds.forEach((id) => next.delete(id));
  } else {
    itemIds.forEach((id) => next.add(id));
  }
  setSelectedEntryIds(next);
};

```

// Toggle penpal selection

```

const handleTogglePenpal = (id: string) => {
  const next = new Set(selectedPenpalIds);
  if (next.has(id)) {
    next.delete(id);
  } else {
    next.add(id);
  }
  setSelectedPenpalIds(next);
};

```

// Push scaffold to selected penpals

```

const handlePush = async () => {
  if (selectedPenpalIds.size === 0) {
    alert('Please select at least one penpal. ');
    return;
  }
  if (selectedEntryIds.size === 0) {
    alert('Please select at least one entry. ');
    return;
  }
};

```

```

}

const pushList = entries.filter((e) => selectedEntryIds.has(e.id));
const scaffoldId = crypto.randomUUID();
const errors: string[] = [];

// 1. Process target penpals one by one
for (const penpalId of selectedPenpalIds) {
  const penpal = penpals.find((p) => p.id === penpalId);
  const penpalName = penpal ? penpal.name : 'Penpal';

  try {
    // Ensure a letter tab exists (either active in store, or open/create draft)
    let tab = openLetters.find((l) => l.penpalId === penpalId);
    let letterId: string;
    if (!tab) {
      // Create new letter draft
      letterId = await openNewLetter(penpalId);
    } else {
      letterId = tab.letterId;
    }

    // 2. Perform duplicate checking against the letter's SQLite contents
    try {
      const currentContent = await loadLetterContent(letterId);
      const existingEntryIds = new Set<string>();

      if (currentContent && currentContent.blocksJson) {
        try {
          const blocks = JSON.parse(currentContent.blocksJson);
          if (Array.isArray(blocks)) {
            blocks.forEach((b: any) => {
              if ((b.type === 'scaffoldBlock' || b.type === 'routedEntry') &&
b.props?.sourceEntryId) {
                existingEntryIds.add(b.props.sourceEntryId);
              }
            });
          }
        } catch (parseErr) {
          console.warn(`[Scaffold Curation] Failed to parse blocksJson for duplicate
detection. Proceeding as if empty:`, parseErr);
        }
      }

      // Check if any push item is already inside the letter
      const duplicates = pushList.filter((item) => existingEntryIds.has(item.id));
      if (duplicates.length > 0) {
        const proceed = confirm(
          `Duplicate Guard Alert!\n\n${duplicates.length} of the selected entries
have already been pushed or routed to ${penpalName}'s letter.\n\nDo you want to push
anyway (this will result in duplicate sections)?`
        );
        if (!proceed) {

```

```

        continue; // Skip this penpal or cancel
    }
}
} catch (err) {
    console.error('Duplicate checking failed:', err);
}

// 3. Queue scaffold blocks via routing queue
// Scaffold sections are grouped by date, so we will assign section labels
matching their dates!
pushList.forEach((entry) => {
    try {
        addToRoutingQueue({
            id: crypto.randomUUID(),
            sourceThoughtUnit: entry,
            targetLetterId: letterId,
            insertPosition: 'end',
            timestamp: Date.now(),
            blockType: 'scaffoldBlock',
            scaffoldId,
            sectionLabel: entry.date || 'Scaffold Section',
        });
    } catch (queueErr: any) {
        console.error(`Failed to enqueue item for ${penpalName}:`, queueErr);
        throw new Error(`Queue addition failed: ${queueErr.message || queueErr}`);
    }
});
} catch (err: any) {
    console.error(`Failed to push scaffold for ${penpalName}:`, err);
    errors.push(`Failed for ${penpalName}: ${err.message || err}`);
}
}

if (errors.length > 0) {
    alert(`Scaffold processing completed with errors:\n\n${errors.join('\n')}`);
} else {
    alert('Scaffold successfully generated and pushed to drafts!');
}
onClose();
};

return (
    <div
        style={{
            position: 'fixed',
            inset: 0,
            background: 'rgba(10, 10, 16, 0.85)',
            display: 'flex',
            justifyContent: 'center',
            alignItems: 'center',
            zIndex: 1000,
            backdropFilter: 'blur(4px)',
        }}
    >

```

```

>
<div
  style={{
    background: 'var(--color-surface-elevated)',
    border: '1px solid var(--color-border)',
    borderRadius: 'var(--radius-lg)',
    width: '90%',
    maxWidth: '850px',
    maxHeight: '90vh',
    display: 'flex',
    flexDirection: 'column',
    boxShadow: 'var(--shadow-lg)',
    overflow: 'hidden',
  }}
>
  {/* Modal Header */}
  <div
    style={{
      padding: 'var(--space-4)',
      borderBottom: '1px solid var(--color-border)',
      display: 'flex',
      justifyContent: 'space-between',
      alignItems: 'center',
    }}
    >
      <div style={{ display: 'flex', alignItems: 'center', gap: 'var(--space-2)' }}>
        <Sparkles size={18} style={{ color: 'var(--color-success, #10b981)' }} />
        <h3 style={{ fontSize: 'var(--text-lg)', fontWeight:
'var(--weight-semibold)', color: 'var(--color-text-primary)' }}>

```

Generate Structured Scaffold

```

  </h3>
</div>
<button
  onClick={onClose}
  style={{
    background: 'none',
    border: 'none',
    color: 'var(--color-text-secondary)',
    cursor: 'pointer',
  }}
  >
    <X size={18} />
  </button>
</div>

  {/* Modal Content */}
  <div style={{ flex: 1, padding: 'var(--space-5)', overflowY: 'auto', display:
'flex', flexDirection: 'column', gap: 'var(--space-4)' }}>
    {/* Step 1: Select Timeframe */}
    <div>
      <h4 style={{ fontSize: 'var(--text-sm)', fontWeight:
'var(--weight-semibold)', color: 'var(--color-text-primary)', marginBottom: '8px' }}>

```


1. Select Timeframe

</h4>

```
<div style={{ display: 'flex', gap: 'var(--space-3)', alignItems: 'center',
flexWrap: 'wrap' }}>
```

```
<div style={{ display: 'flex', gap: '8px', alignItems: 'center' }}>
```

```
<Calendar size={14} style={{ color: 'var(--color-text-secondary)' }} />
```

```
<input
```

```
  type="date"
```

```
  value={startDate}
```

```
  onChange={(e) => setStartDate(e.target.value)}
```

```
  style={{
```

```
    background: 'var(--color-surface)',
```

```
    border: '1px solid var(--color-border)',
```

```
    borderRadius: 'var(--radius-md)',
```

```
    padding: 'var(--space-2)',
```

```
    color: 'var(--color-text-primary)',
```

```
    fontSize: 'var(--text-xs)',
```

```
  }}>
```

```
/>
```

```
<span style={{ color: 'var(--color-text-secondary)' }}>to</span>
```

```
<input
```

```
  type="date"
```

```
  value={endDate}
```

```
  onChange={(e) => setEndDate(e.target.value)}
```

```
  style={{
```

```
    background: 'var(--color-surface)',
```

```
    border: '1px solid var(--color-border)',
```

```
    borderRadius: 'var(--radius-md)',
```

```
    padding: 'var(--space-2)',
```

```
    color: 'var(--color-text-primary)',
```

```
    fontSize: 'var(--text-xs)',
```

```
  }}>
```

```
/>
```

```
</div>
```

```
{/* Quick Presets */}
```

```
<div style={{ display: 'flex', gap: '6px' }}>
```

```
<button
```

```
  type="button"
```

```
  onClick={() => handlePreset('week')}
```

```
  style={{
```

```
    background: 'var(--color-surface)',
```

```
    border: '1px solid var(--color-border)',
```

```
    borderRadius: 'var(--radius-sm)',
```

```
    fontSize: '10px',
```

```
    padding: '4px 8px',
```

```
    cursor: 'pointer',
```

```
    color: 'var(--color-text-primary)',
```

```
  }}>
```

```
>
```

```
  This Week
```

```
</button>
```

```
<button
```

```

        type="button"
        onClick={() => handlePreset('month')}
        style={{
            background: 'var(--color-surface)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-sm)',
            fontSize: '10px',
            padding: '4px 8px',
            cursor: 'pointer',
            color: 'var(--color-text-primary)',
        }}
    >
        This Month
    </button>
</div>

<button
    onClick={handleFetch}
    disabled={loading}
    style={{
        background: 'var(--color-primary)',
        color: 'white',
        border: 'none',
        borderRadius: 'var(--radius-md)',
        padding: 'var(--space-2) var(--space-4)',
        fontSize: 'var(--text-xs)',
        fontWeight: 'var(--weight-semibold)',
        cursor: 'pointer',
        marginLeft: 'auto',
    }}
>
    {loading ? 'Fetching...' : 'Fetch Journal Entries'}
</button>
</div>
</div>

{/* Columns for Preview and Penpals */}
<div style={{ display: 'flex', gap: 'var(--space-4)', flex: 1, minHeight:
'300px' }}>
    {/* Step 2: Curation checklist */}
    <div
        style={{
            flex: 2,
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-md)',
            background: 'var(--color-surface)',
            display: 'flex',
            flexDirection: 'column',
            overflow: 'hidden',
        }}
    >
        <div
            style={{

```

```

padding: 'var(--space-3)',
borderBottom: '1px solid var(--color-border)',
background: 'var(--color-surface-elevated)',
fontSize: 'var(--text-xs)',
fontWeight: 'var(--weight-bold)',
color: 'var(--color-text-primary)',
}}
>
2. Curate Scaffold Thoughts ({selectedEntryIds.size} selected)
</div>
<div
  ref={parentRef}
  style={{
    flex: 1,
    overflowY: 'auto',
    padding: 'var(--space-3)',
  }}
>
  {loading ? (
    <div style={{ padding: '40px', textAlign: 'center', color:
'var(--color-text-secondary)' }}>Loading entries...</div>
    ) : !fetched ? (
      <div style={{ padding: '40px', textAlign: 'center', color:
'var(--color-text-secondary)', fontSize: 'var(--text-xs)' }}>
        Select a date range and click <strong>Fetch Journal Entries</strong>
        above to load thoughts.
      </div>
    ) : entries.length === 0 ? (
      <div style={{ padding: '40px', textAlign: 'center', color:
'var(--color-text-secondary)', fontSize: 'var(--text-xs)' }}>
        No journal entries found for this timeframe. Try a different range.
      </div>
    ) : (
      <div
        style={{
          height: `${rowVirtualizer.getTotalSize()}px`,
          width: '100%',
          position: 'relative',
        }}
      >
        {rowVirtualizer.getVirtualItems().map((virtualRow) => {
          const item = flatList[virtualRow.index];
          if (!item) return null;
          if (item.type === 'header') {
            const groupIds = item.items!.map((i) => i.id);
            const allSelectedInGroup = groupIds.every((id) =>
selectedEntryIds.has(id));
            return (
              <div
                key={virtualRow.key}
                style={{
                  position: 'absolute',
                  top: 0,

```

```

        left: 0,
        width: '100%',
        height: `${virtualRow.size}px`,
        transform: `translateY(${virtualRow.start}px)`,
        padding: '2px 0',
    }}
>
<div
  style={{
    display: 'flex',
    alignItems: 'center',
    gap: '8px',
    fontWeight: 'var(--weight-semibold)',
    color: 'var(--color-primary)',
    fontSize: 'var(--text-xs)',
    borderBottom: '1px solid var(--color-border)',
    paddingBottom: '4px',
  }}
>
  <input
    type="checkbox"
    checked={allSelectedInGroup}
    onChange={() => handleToggleDate(item.items!)}
    style={{ cursor: 'pointer' }}
  />
  <span>{item.date}</span>
  <span style={{ fontSize: '10px', background: 'rgba(2, 132,
199, 0.1)', padding: '0 4px', borderRadius: 'var(--radius-sm)' }}>
    {item.items!.length} entry{item.items!.length > 1 ? 's'
: ''}

  </span>
</div>
</div>
);
} else {
  const tu = item.item!;
  return (
    <div
      key={virtualRow.key}
      style={{
        position: 'absolute',
        top: 0,
        left: 0,
        width: '100%',
        height: `${virtualRow.size}px`,
        transform: `translateY(${virtualRow.start}px)`,
        padding: '3px 0',
      }}
    >
      <label
        style={{
          display: 'flex',
          alignItems: 'flex-start',

```

```

        gap: '8px',
        padding: 'var(--space-2)',
        background: 'var(--color-surface-elevated)',
        border: '1px solid var(--color-border)',

        borderRadius: 'var(--radius-sm)',
        cursor: 'pointer',
        fontSize: 'var(--text-xs)',
        color: 'var(--color-text-primary)',
        height: '100%',
        boxSizing: 'border-box',
        overflow: 'hidden',
    }}
    >
    <input
        type="checkbox"
        checked={selectedEntryIds.has(tu.id)}
        onChange={() => handleToggleEntry(tu.id)}
        style={{ marginTop: '2px', cursor: 'pointer' }}
    />
    <div style={{ flex: 1, height: '100%', overflow: 'hidden',
display: 'flex', flexDirection: 'column' }}>
        <div style={{ display: 'flex', gap: '6px', marginBottom:
'2px', flexShrink: 0 }}>
            <span style={{ fontSize: '9px', textTransform:
'uppercase', padding: '0 4px', borderRadius: 'var(--radius-sm)', background: tu.category
=== 'presence' ? 'rgba(34,197,94,0.1)' : tu.category === 'reminiscence' ?
'rgba(139,92,246,0.1)' : 'rgba(100,116,139,0.1)', color: tu.category === 'presence' ?
'#10b981' : tu.category === 'reminiscence' ? '#8b5cf6' : '#64748b' }}>
                #{tu.category}
            </span>
            <span style={{ fontSize: '9px', color:
'var(--color-text-secondary)' }}>Line {tu.sourceLineNumber}</span>
        </div>
        <div style={{ flex: 1, whiteSpace: 'pre-wrap',
lineHeight: '1.4', overflowY: 'auto' }}>{tu.content}</div>
    </div>
</label>
</div>
    );
    }
    }}}
</div>
)}
</div>
</div>

{/* Step 3: Select Penpals */}
<div
    style={{
        flex: 1,
        border: '1px solid var(--color-border)',
        borderRadius: 'var(--radius-md)',

```

```

        background: 'var(--color-surface)',
        display: 'flex',
        flexDirection: 'column',
        overflow: 'hidden',
    }}
>
<div
  style={{
    padding: 'var(--space-3)',
    borderBottom: '1px solid var(--color-border)',
    background: 'var(--color-surface-elevated)',
    fontSize: 'var(--text-xs)',
    fontWeight: 'var(--weight-bold)',
    color: 'var(--color-text-primary)',
  }}
>
  3. Select Target Penpals ({selectedPenpalIds.size} selected)
</div>
  <div style={{ flex: 1, overflowY: 'auto', padding: 'var(--space-3)',
display: 'flex', flexDirection: 'column', gap: '8px' }}>
    {penpals.length === 0 ? (
      <div style={{ padding: '20px', textAlign: 'center', color:
'var(--color-text-secondary)', fontSize: 'var(--text-xs)' }}>
        No penpals added yet. Go to Penpal Panel to add one!
      </div>
    ) : (
      penpals.map((p) => (
        <label
          key={p.id}
          style={{
            display: 'flex',
            alignItems: 'center',
            gap: '8px',
            padding: '8px var(--space-3)',
            background: 'var(--color-surface-elevated)',
            border: '1px solid var(--color-border)',
            borderRadius: 'var(--radius-sm)',
            cursor: 'pointer',
            fontSize: 'var(--text-xs)',
            color: 'var(--color-text-primary)',
          }}
        >
          <input
            type="checkbox"
            checked={selectedPenpalIds.has(p.id)}
            onChange={() => handleTogglePenpal(p.id)}
            style={{ cursor: 'pointer' }}
          />
          <div style={{ flex: 1 }}>
            <div style={{ fontWeight: 'var(--weight-semibold)'
}}>{p.name}</div>
            <div style={{ fontSize: '10px', color:
'var(--color-text-secondary)' }}>{p.country || 'No Country'}</div>

```

```

        </div>
      </label>
    ))
  })
</div>
</div>
</div>
</div>

{/* Modal Footer */}
<div
  style={{
    padding: 'var(--space-4)',
    borderTop: '1px solid var(--color-border)',
    background: 'var(--color-surface)',
    display: 'flex',
    justifyContent: 'flex-end',
    gap: 'var(--space-3)',
  }}
>
  <button
    onClick={onClose}
    style={{
      background: 'var(--color-surface-elevated)',
      border: '1px solid var(--color-border)',
      borderRadius: 'var(--radius-md)',
      padding: 'var(--space-2) var(--space-4)',
      color: 'var(--color-text-primary)',
      fontSize: 'var(--text-xs)',
      cursor: 'pointer',
    }}
  >
    Cancel
  </button>
  <button
    onClick={handlePush}
    disabled={selectedEntryIds.size === 0 || selectedPenpalIds.size === 0}
    style={{
      background: 'var(--color-success, #10b981)',
      color: 'white',
      border: 'none',
      borderRadius: 'var(--radius-md)',
      padding: 'var(--space-2) var(--space-4)',
      fontSize: 'var(--text-xs)',
      fontWeight: 'var(--weight-semibold)',
      cursor: 'pointer',
      display: 'flex',
      alignItems: 'center',
      gap: '6px',
      opacity: selectedEntryIds.size === 0 || selectedPenpalIds.size === 0 ? 0.5
: 1,
    }}
  >

```

```

        <Check size={14} />
        Push Structured Scaffold
    </button>
</div>
</div>
</div>
);
};

```

--- END_FILE ---

--- START_FILE: src/components/WorkspacePanel.tsx ---

```

import React, { useEffect, useState, useRef } from 'react';
import { useAppStore } from '../stores/appStore';
import { JournalSourcesView } from '../JournalSourcesView';
import { LetterEditor } from '../LetterEditor';
import { ScaffoldModal } from '../ScaffoldModal';
import { markLetterSent, saveLetterContent } from '../services/letterService';
import { addCorrespondence } from '../services/penpalService';
import { serializeToPlainText } from '../utils/serialization';
import { writeText } from '@tauri-apps/plugin-clipboard-manager';
import { X, FileText, Mail, Check, Copy, Sparkles } from 'lucide-react';

interface WorkspacePanelProps {
  showSources: boolean;
  onCloseSources: () => void;
  onOpenSources: () => void;
}

export const WorkspacePanel: React.FC<WorkspacePanelProps> = ({
  showSources,
  onCloseSources,
  onOpenSources,
}) => {
  const { openLetters, activeLetterIdx, closeLetterTab, penpals } = useAppStore();

  const [saving, setSaving] = useState<boolean>(false);
  const [lastSaved, setLastSaved] = useState<string>('');
  const [showScaffoldModal, setShowScaffoldModal] = useState<boolean>(false);

  // Lazy tab mounting state: contains at most 3 letterIds currently alive in DOM
  const [aliveLetterIds, setAliveLetterIds] = useState<string[]>([]);

  // Keep track of blocks in the active editor instances (to serialize/copy without
  re-renders)
  const activeBlocksMapRef = useRef<Record<string, any[]>>({});

  const activeLetter = activeLetterIdx >= 0 && activeLetterIdx < openLetters.length ?
  openLetters[activeLetterIdx] : null;

```



```

// 1. Maintain MRU list of alive letter tabs (maximum 3 in DOM at once)
useEffect(() => {
  if (!activeLetter) return;

  setAliveLetterIds((prev) => {
    const openIds = openLetters.map((l) => l.letterId);
    const filteredPrev = prev.filter((id) => openIds.includes(id));

    if (filteredPrev[0] === activeLetter.letterId) {
      return filteredPrev;
    }

    const next = [
      activeLetter.letterId,
      ...filteredPrev.filter((id) => id !== activeLetter.letterId),
    ];

    return next.slice(0, 3);
  });

  onCloseSources(); // Hide sources if we switched to a draft letter tab
}, [activeLetter, openLetters]);

// 1.5. Tauri Window Close Interceptor: flush all dirty active letters before closing
useEffect(() => {
  let unlisten: (() => void) | null = null;

  const setupCloseInterceptor = async () => {
    try {
      const { getCurrentWindow } = await import('@tauri-apps/api/window');
      const win = getCurrentWindow();

      unlisten = await win.onCloseRequested(async (event) => {
        // Prevent standard abrupt close
        event.preventDefault();
        setSaving(true);

        try {
          // Save all letters that have active editor content in memory
          const savePromises = openLetters.map(async (tab) => {
            const blocks = activeBlocksMapRef.current[tab.letterId];
            if (blocks && Array.isArray(blocks)) {
              const content = JSON.stringify(blocks);
              await saveLetterContent(tab.letterId, content);
            }
          });

          await Promise.all(savePromises);
        } catch (err) {
          console.error('Failed to flush letters on close:', err);
        }
      });

      // Once flushed, destroy the window to terminate
    }
  };

```

```

        await win.destroy();
    });
} catch (err) {
    console.error('Failed to bind window close interceptor:', err);
}
};

setupCloseInterceptor();

return () => {
    if (unlisten) {
        unlisten();
    }
};
}, [openLetters]));

// 2. Clipboard copy action (stripping markdown metadata/badges)
const handleCopyToClipboard = async () => {
    if (!activeLetter) return;
    const blocks = activeBlocksMapRef.current[activeLetter.letterId] || [];
    const plainText = serializeToPlainText(blocks);

    if (!plainText.trim()) {
        alert('Letter is empty. Add some content first.');
```

return;

```
    }

    try {
        await writeText(plainText);
        alert('Letter copied to clipboard successfully!');
```

} catch (err) {

```
    console.error('Failed to copy text:', err);
    alert('Failed to copy to clipboard.');
```

}

```
};

// 3. Archive & Mark as sent action (with non-empty validation)
const handleMarkSent = async () => {
    if (!activeLetter) return;
    const penpal = penpals.find((p) => p.id === activeLetter.penpalId);
    const penpalName = penpal ? penpal.name : 'Penpal';

    const blocks = activeBlocksMapRef.current[activeLetter.letterId] || [];
    const plainText = serializeToPlainText(blocks);

    if (!plainText.trim()) {
        alert('Cannot mark an empty letter as sent. Add some content first.');
```

return;

```
    }

    if (
        !confirm(
            `Are you sure you want to mark this letter to ${penpalName} as SENT? This will
```

```

archive it in your correspondence history and close the tab.`
    )
  ) {
    return;
  }

  try {
    // 1. Archive sent content to correspondence table
    await addCorrespondence(
      activeLetter.penpalId,
      'sent',
      plainText,
      new Date().toISOString().split('T')[0] || ''
    );

    // 2. Mark draft status as sent in database
    await markLetterSent(activeLetter.letterId);

    // 3. Close the letter workspace tab
    closeLetterTab(activeLetter.letterId);
    alert('Letter successfully archived and closed!');
  } catch (err: any) {
    alert(`Failed to mark letter as sent: ${err}`);
  }
};

// Get matching penpal name for active letter tab
const getPenpalName = (penpalId: string) => {
  const penpal = penpals.find((p) => p.id === penpalId);
  return penpal ? penpal.name : 'Unknown';
};

return (
  <div style={{ display: 'flex', flexDirection: 'column', height: '100%', background:
'var(--color-background)' }}>
    /* Workspace Tabs Header Bar */
    <div
      style={{
        display: 'flex',
        background: 'var(--color-surface)',
        borderBottom: '1px solid var(--color-border)',
        overflowX: 'auto',
        alignItems: 'center',
        width: '100%',
      }}
    >
      /* Special Journal Sources Tab */
      <button
        onClick={onOpenSources}
        style={{
          display: 'flex',
          alignItems: 'center',
          gap: 'var(--space-2)',

```

```

padding: 'var(--space-3) var(--space-4)',
background: showSources ? 'var(--color-background)' :
'var(--color-surface)',
color: showSources ? 'var(--color-primary)' : 'var(--color-text-secondary)',
border: 'none',
borderRight: '1px solid var(--color-border)',
cursor: 'pointer',
fontSize: 'var(--text-sm)',
fontWeight: 'var(--weight-semibold)',
borderTop: showSources ? '2px solid var(--color-primary)' : 'none',
flexShrink: 0,
}}
>
<FileText size={14} />
Journal Sources
</button>

{/* Dynamic Letter Tabs */}
{openLetters.map((tab, idx) => {
  const isActive = !showSources && idx === activeLetterIdx;
  return (
    <div
      key={tab.letterId}
      style={{
        display: 'flex',
        alignItems: 'center',
background: isActive ? 'var(--color-background)' :
'var(--color-surface)',
borderRight: '1px solid var(--color-border)',
borderTop: isActive ? '2px solid var(--color-primary)' : 'none',
flexShrink: 0,
      }}
    >
      <button
        onClick={() => {
          onCloseSources();
          useAppStore.setState({ activeLetterIdx: idx });
        }}
        style={{
          display: 'flex',
          alignItems: 'center',
          gap: 'var(--space-2)',
          padding: 'var(--space-3) 8px var(--space-3) var(--space-4)',
color: isActive ? 'var(--color-text-primary)' :
'var(--color-text-secondary)',
          background: 'none',
          border: 'none',
          cursor: 'pointer',
          fontSize: 'var(--text-sm)',
          fontWeight: 'var(--weight-medium)',
        }}
      >
        <Mail size={14} style={{ color: isActive ? 'var(--color-primary)' :

```

```

'inherit' }} />
    Letter to {getPenpalName(tab.penpalId)}
  </button>
  <button
    onClick={() => closeLetterTab(tab.letterId)}
    style={{
      background: 'none',
      border: 'none',
      color: 'var(--color-text-secondary)',
      cursor: 'pointer',
      padding: '4px',
      marginRight: '8px',
      borderRadius: 'var(--radius-sm)',
      display: 'flex',
      alignItems: 'center',
    }}
    onMouseOver={(e) => (e.currentTarget.style.color =
'var(--color-error)')}
    onMouseOut={(e) => (e.currentTarget.style.color =
'var(--color-text-secondary)')}
  >
    <X size={12} />
  </button>
</div>
);
}}

{/* Create Scaffold Modal Action Button */}
<button
  onClick={() => setShowScaffoldModal(true)}
  style={{
    display: 'flex',
    alignItems: 'center',
    gap: 'var(--space-2)',
    padding: 'var(--space-3) var(--space-4)',
    background: 'var(--color-surface)',
    color: 'var(--color-text-secondary)',
    border: 'none',
    borderLeft: '1px solid var(--color-border)',
    cursor: 'pointer',
    fontSize: 'var(--text-sm)',
    fontWeight: 'var(-
-weight-semibold)',
    marginLeft: 'auto',
    flexShrink: 0,
  }}
  onMouseOver={(e) => (e.currentTarget.style.color = 'var(--color-success,
#10b981)')}
  onMouseOut={(e) => (e.currentTarget.style.color =
'var(--color-text-secondary)')}
>
  <Sparkles size={14} style={{ color: '#10b981' }} />
  Create Scaffold

```

```

    </button>
  </div>

  { /* Main Workspace Body Content */ }
  <div style={{ flex: 1, overflowY: 'auto' }}>
    {showSources ? (
      <JournalSourcesView />
    ) : activeLetter ? (
      <div style={{ padding: 'var(--space-6)', display: 'flex', flexDirection:
'column', gap: 'var(--space-4)', height: '100%' }}>
        { /* Metadata Bar */ }
        <div style={{ display: 'flex', justifyContent: 'space-between', alignItems:
'center' }}>
          <div>
            <h2 style={{ fontSize: 'var(--text-xl)', fontWeight:
'var(--weight-bold)', color: 'var(--color-text-primary)' }}>
              Drafting Letter for {getPenpalName(activeLetter.penpalId)}
            </h2>
            <span style={{ fontSize: 'var(--text-xs)', color:
'var(--color-text-secondary)' }}>
              {lastSaved} {saving && ' • Auto-saving...'}
            </span>
          </div>
        </div>

        <div style={{ display: 'flex', gap: 'var(--space-2)' }}>
          <button
            onClick={handleCopyToClipboard}
            style={{
              background: 'var(--color-surface-elevated)',
              color: 'var(--color-primary)',
              border: '1px solid var(--color-border)',
              borderRadius: 'var(--radius-md)',
              padding: 'var(--space-2) var(--space-4)',
              fontSize: 'var(--text-sm)',
              fontWeight: 'var(--weight-semibold)',
              cursor: 'pointer',
              display: 'flex',
              alignItems: 'center',
              gap: 'var(--space-2)',
            }}
            onMouseOver={{(e) => (e.currentTarget.style.borderColor =
'var(--color-primary))'}}
            onMouseOut={{(e) => (e.currentTarget.style.borderColor =
'var(--color-border))'}}
          >
            <Copy size={16} />
            Copy Letter
          </button>

          <button
            onClick={handleMarkSent}
            style={{
              background: 'var(--color-success)',

```

```

        color: 'white',
        border: 'none',
        borderRadius: 'var(--radius-md)',
        padding: 'var(--space-2) var(--space-4)',
        fontSize: 'var(--text-sm)',
        fontWeight: 'var(--weight-semibold)',
        cursor: 'pointer',
        display: 'flex',
        alignItems: 'center',
        gap: 'var(--space-2)',
    }}
    >
    <Check size={16} />
    Mark as Sent
  </button>
</div>
</div>

```

```

        /* Dynamic Editors (Only rendering MRU alive editors, others
hidden/unmounted) */}
    <div style={{ flex: 1, display: 'flex', flexDirection: 'column', position:
'relative' }}>
      {openLetters.map((tab, idx) => {
        const isAlive = aliveLetterIds.includes(tab.letterId);
        if (!isAlive) return null;

        const isActive = !showSources && idx === activeLetterIdx;
        return (
          <div
            key={tab.letterId}
            style={{
              display: isActive ? 'flex' : 'none',
              flexDirection: 'column',
              flex: 1,
              width: '100%',
            }}
          >
            <LetterEditor
              letterId={tab.letterId}
              onBlocksChange={(blocks) => {
                activeBlocksMapRef.current[tab.letterId] = blocks;
              }}
              onSaveStateChange={(savingVal, msg) => {
                if (isActive) {
                  setSaving(savingVal);
                  setLastSaved(msg);
                }
              }}
            />
          </div>
        );
      })}
    </div>

```

```

    </div>
  ) : (
    /* Welcome screen when workspace is empty */
    <div style={{ display: 'flex', flexDirection: 'column', justifyContent:
'center', alignItems: 'center', height: '100%', padding: 'var(--space-12)', textAlign:
'center', color: 'var(--color-text-secondary)' }}>
      <FileText size={64} style={{ marginBottom: 'var(--space-4)', color:
'var(--color-primary)', opacity: 0.8 }} />
      <h2 style={{ fontSize: 'var(--text-xl)', fontWeight: 'var(--weight-bold)',
color: 'var(--color-text-primary)' }}>
        Journal-to-Penpal Pipeline
      </h2>
      <p style={{ maxWidth: '400px', fontSize: 'var(--text-sm)', marginTop:
'var(--space-2)', lineHeight: 'var(--leading-sm)' }}>
        Start writing by choosing a penpal on the right and clicking <strong>Write
Letter</strong>, or manage your <strong>Journal Sources</strong> to scan daily entries.
      </p>
    </div>
  )}
</div>

{/* Scaffold Modal Render */}
{showScaffoldModal && (
  <ScaffoldModal onClose={() => setShowScaffoldModal(false)} />
)}
</div>
);
};

```

--- END_FILE ---

--- START_FILE: src/components/blocks/schema.tsx ---

```

import { createReactBlockSpec } from "@blocknote/react";
import { BlockNoteSchema, defaultBlockSpecs } from "@blocknote/core";

export const ROUTED_ENTRY_TYPE = "routedEntry" as const;

export const RoutedEntry = createReactBlockSpec(
  {
    type: ROUTED_ENTRY_TYPE,
    propSchema: {
      sourceEntryId: { default: "" },
      sourceDate: { default: "" },
      category: { default: "uncategorized",
        values: ["presence", "reminiscence", "uncategorized"] },
    },
    content: "inline",
  },
  {
    render: ({ block, contentRef }) => {

```



```

const category = block.props.category || "uncategorized";
const categoryColor =
  category === "presence"
    ? "var(--color-presence, #4ECDC4)"
    : category === "reminiscence"
    ? "var(--color-reminiscence, #FF8A65)"
    : "var(--color-primary, #7C6AEF)";
const categoryBg =
  category === "presence"
    ? "var(--color-presence-tint, rgba(78, 205, 196, 0.08))"
    : category === "reminiscence"
    ? "var(--color-reminiscence-tint, rgba(255, 138, 101, 0.08))"
    : "var(--color-scaffold-tint, rgba(124, 106, 239, 0.08))";

return (
  <div
    className="routed-entry-block-container"
    style={{
      borderLeft: `4px solid ${categoryColor}`,
      background: categoryBg,
      padding: "8px 12px",
      borderRadius: "0 var(--radius-md) var(--radius-md) 0",
      margin: "6px 0",
      width: "100%",
    }}
  >
    <div
      className="provenance-badge"
      contentEditable={false}
      style={{
        fontSize: "10px",
        color: categoryColor,
        fontWeight: "var(--weight-semibold)" as any,
        marginBottom: "4px",
        textTransform: "uppercase",
        letterSpacing: "0.05em",
        userSelect: "none",
      }}
    >
      {block.props.sourceDate || "Undated"} · #{block.props.category}
    </div>
    <div ref={contentRef} style={{ fontSize: "var(--text-base)", color:
"var(--color-text-primary)" }} />
  </div>
);
},
},
);

export const ScaffoldBlock = createReactBlockSpec(
  {
    type: "scaffoldBlock",
    propSchema: {

```

```

    scaffoldId: { default: "" },
    sectionLabel: { default: "" },
    sourceEntryId: { default: "" },
  },
  content: "inline",
},
{
  render: ({ block, contentRef }) => {
    const sectionLabel = String(block.props.sectionLabel ?? "Scaffold Section");
    return (
      <div
        className="scaffold-block-container"
        style={{
          borderLeft: "4px solid #10b981",
          background: "rgba(16, 185, 129, 0.08)",
          padding: "8px 12px",
          borderRadius: "0 var(--radius-md) var(--radius-md) 0",
          margin: "6px 0",
          width: "100%",
        }}
      >
        <div
          className="scaffold-label"
          contentEditable={false}
          style={{
            fontSize: "10px",
            color: "#10b981",
            fontWeight: "var(--weight-semibold)" as any,
            marginBottom: "4px",
            textTransform: "uppercase",
            letterSpacing: "0.05em",
            userSelect: "none",
          }}
        >
          Scaffold: {sectionLabel}
        </div>
        <div ref={contentRef} style={{ fontSize: "var(--text-base)", color:
"var(--color-text-primary)" }} />
      </div>
    );
  },
},
);

export const schema = BlockNoteSchema.create({
  blockSpecs: {
    ...defaultBlockSpecs,
    routedEntry: RoutedEntry(),
    scaffoldBlock: ScaffoldBlock(),
  },
});

```

--- END_FILE ---

--- START_FILE: src/main.tsx ---

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "../App";
import "../styles/global.css";

ReactDOM.createRoot(document.getElementById("root") as HTMLElement).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

--- END_FILE ---

--- START_FILE: src/services/journalService.ts ---

```
import { invoke } from '@tauri-apps/api/core';
import { ThoughtUnit, JournalSource, ImportResult } from '../types';

export async function importJournalFile(filePath: string): Promise<ImportResult> {
  return invoke<ImportResult>('import_journal_file', { filePath });
}

export async function getImportStatus(): Promise<JournalSource[]> {
  return invoke<JournalSource[]>('get_import_status');
}

export async function removeJournalSource(filePath: string): Promise<boolean> {
  return invoke<boolean>('remove_journal_source', { filePath });
}

export async function getThoughtUnits(
  startDate?: string | null,
  endDate?: string | null,
  category?: string | null
): Promise<ThoughtUnit[]> {
  return invoke<ThoughtUnit[]>('get_thought_units', {
    startDate: startDate || null,
    endDate: endDate || null,
    category: category || null,
  });
}
```

--- END_FILE ---

```
--- START_FILE: src/services/letterService.ts ---
```

```
import { invoke } from '@tauri-apps/api/core';
import { Letter } from '../types';
```

```
export async function createLetter(penpalId: string): Promise<Letter> {
  return invoke<Letter>('create_letter', { penpalId });
}
```

```
export async function saveLetterContent(letterId: string, blocksJson: string):
Promise<boolean> {
  return invoke<boolean>('save_letter_content', { letterId, blocksJson });
}
```

```
export async function loadLetterContent(letterId: string): Promise<{ blocksJson: string
}> {
  return invoke<{ blocksJson: string }>('load_letter_content', { letterId });
}
```

```
export async function markLetterSent(letterId: string): Promise<boolean> {
  return invoke<boolean>('mark_letter_sent', { letterId });
}
```

```
--- END_FILE ---
```

```
--- START_FILE: src/services/penpalService.ts ---
```

```
import { invoke } from '@tauri-apps/api/core';
import { Penpal, Correspondence } from '../types';
```

```
export async function createPenpal(
  name: string,
  country?: string,
  interests?: string,
  topics?: string,
  notes?: string
): Promise<Penpal> {
  return invoke<Penpal>('create_penpal', {
    name,
    country: country || null,
    interests: interests || null,
    topics: topics || null,
    notes: notes || null,
  });
}
```

```
export async function getPenpals(): Promise<Penpal[]> {
  return invoke<Penpal[]>('get_penpals');
}
```

```

export async function updatePenpal(
  id: string,
  name: string,
  country?: string,
  interests?: string,
  topics?: string,
  notes?: string
): Promise<Penpal> {
  return invoke<Penpal>('update_penpal', {
    id,
    name,
    country: country || null,
    interests: interests || null,
    topics: topics || null,
    notes: notes || null,
  });
}

export async function addCorrespondence(
  penpalId: string,
  direction: 'sent' | 'received',
  content: string,
  letterDate: string
): Promise<Correspondence> {
  return invoke<Correspondence>('add_correspondence', {
    penpalId,
    direction,
    content,
    letterDate,
  });
}

export async function getCorrespondence(penpalId: string): Promise<Correspondence[]> {
  return invoke<Correspondence[]>('get_correspondence', { penpalId });
}

--- END_FILE ---

--- START_FILE: src/services/workspaceService.ts ---

import { invoke } from '@tauri-apps/api/core';
import { WorkspaceState } from '../types';

export async function saveWorkspaceState(
  openLetterIds: string,
  activeLetterId: string | null,
  crawlerState: string
): Promise<boolean> {
  return invoke<boolean>('save_workspace_state', {
    openLetterIds,
    activeLetterId,
  });
}

```

```

        crawlerState,
    });
}

export async function loadWorkspaceState(): Promise<WorkspaceState> {
    return invoke<WorkspaceState>('load_workspace_state');
}

export async function getPersistValue(key: string): Promise<string | null> {
    return invoke<string | null>('get_persist_value', { key });
}

export async function setPersistValue(key: string, value: string): Promise<boolean> {
    return invoke<boolean>('set_persist_value', { key, value });
}

export async function deletePersistValue(key: string): Promise<boolean> {
    return invoke<boolean>('delete_persist_value', { key });
}

--- END_FILE ---

--- START_FILE: src/stores/appStore.ts ---

import { create } from 'zustand';
import { persist, createJSONStorage, StateStorage } from 'zustand/middleware';
import { Penpal, RoutingQueueItem } from '../types';
import { getPenpals } from '../services/penpalService';
import { createLetter } from '../services/letterService';
import {
    getPersistValue,
    setPersistValue,
    deletePersistValue,
    saveWorkspaceState,
    loadWorkspaceState,
} from '../services/workspaceService';

// Custom SQLite persist adapter for Zustand
const sqliteStorage: StateStorage = {
    getItem: async (name) => {
        try {
            const rawPersist = await getPersistValue(name);
            if (!rawPersist) return null;

            const parsed = JSON.parse(rawPersist);
            if (!parsed || !parsed.state) return rawPersist;

            // Fetch cleaned workspace state from relational table
            const workspace = await loadWorkspaceState();
            const validOpenLetterIds: string[] = JSON.parse(workspace.openLetterIds || '[]');

```

```

        const openLetters: { letterId: string; penpalId: string }[] =
parsed.state.openLetters || [];
        const validLetters = openLetters.filter((l) =>
validOpenLetterIds.includes(l.letterId));

        // Re-calculate active tab index
        let activeIdx = parsed.state.activeLetterIdx;
        if (workspace.activeLetterId) {
            activeIdx = validLetters.findIndex((l) => l.letterId ===
workspace.activeLetterId);
        } else {
            activeIdx = validLetters.length > 0 ? 0 : -1;
        }

        parsed.state.openLetters = validLetters;
        parsed.state.activeLetterIdx = activeIdx;

        return JSON.stringify(parsed);
    } catch (e) {
        console.error('Failed to load workspace state with cleanup:', e);
        return getPersistValue(name);
    }
},
setItem: async (name, value) => {
    try {
        await setPersistValue(name, value);

        const parsed = JSON.parse(value);
        if (parsed && parsed.state) {
            const openLetters: { letterId: string; penpalId: string }[] =
parsed.state.openLetters || [];
            const activeIdx: number = parsed.state.activeLetterIdx;
            const crawlerFilters = parsed.state.crawlerFilters || {};

            const openLetterIds = openLetters.map((l) => l.letterId);
            const activeLetterId = activeIdx >= 0 && activeIdx < openLetters.length
                ? openLetters[activeIdx]?.letterId || null
                : null;

            await saveWorkspaceState(
                JSON.stringify(openLetterIds),
                activeLetterId,
                JSON.stringify(crawlerFilters)
            );
        }
    } catch (e) {
        console.error('Failed to save relational workspace state:', e);
    }
},
removeItem: async (name) => {
    await deletePersistValue(name);
    try {
        await saveWorkspaceState('[]', null, '{}');
    }
}

```

```

    } catch (e) {
      console.error('Failed to clear relational workspace state:', e);
    }
  },
};

interface CrawlerFilters {
  startDate: string | null;
  endDate: string | null;
  category: 'presence' | 'reminiscence' | 'uncategorized' | null;
  searchQuery: string;
}

interface AppStore {
  // Penpal State
  penpals: Penpal[];
  loadPenpals: () => Promise<void>;
  setPenpals: (penpals: Penpal[]) => void;

  // Workspace Tabs State
  openLetters: { letterId: string; penpalId: string }[];
  activeLetterIdx: number;
  openNewLetter: (penpalId: string) => Promise<string>; // Returns letterId
  closeLetterTab: (letterId: string) => void;

  // Routing Queue
  routingQueue: RoutingQueueItem[];
  addToRoutingQueue: (item: RoutingQueueItem) => void;
  removeFromRoutingQueue: (itemId: string) => void;

  // Crawler State
  crawlerFilters: CrawlerFilters;
  setCrawlerFilters: (filters: Partial<CrawlerFilters>) => void;
}

export const useAppStore = create<AppStore>()(
  persist(
    (set, get) => ({
      // Penpal State
      penpals: [],
      loadPenpals: async () => {
        try {
          const list = await getPenpals();
          set({ penpals: list });
        } catch (e) {
          console.error('Failed to load penpals in store:', e);
        }
      },
      setPenpals: (penpals) => set({ penpals }),

      // Workspace Tabs State
      openLetters: [],
      activeLetterIdx: -1,
    })
  )
);

```



```

openNewLetter: async (penpalId: string) => {
  const existingIdx = get().openLetters.findIndex((l) => l.penpalId === penpalId);
  if (existingIdx !== -1) {
    const existing = get().openLetters[existingIdx];
    if (existing) {
      set({ activeLetterIdx: existingIdx });
      return existing.letterId;
    }
  }

  // 2. Fetch or create draft letter from DB
  const letter = await createLetter(penpalId);
  const newOpenLetters = [...get().openLetters, { letterId: letter.id, penpalId
}];

  set({
    openLetters: newOpenLetters,
    activeLetterIdx: newOpenLetters.length - 1,
  });
  return letter.id;
},
closeLetterTab: (letterId: string) => {
  const currentOpen = get().openLetters;
  const currentIdx = get().activeLetterIdx;
  const targetIdx = currentOpen.findIndex((l) => l.letterId === letterId);

  if (targetIdx === -1) return;

  const nextOpen = currentOpen.filter((l) => l.letterId !== letterId);
  let nextIdx = currentIdx;

  if (nextOpen.length === 0) {
    nextIdx = -1;
  } else if (currentIdx >= nextOpen.length) {
    nextIdx = nextOpen.length - 1;
  } else if (targetIdx <= currentIdx && currentIdx > 0) {
    nextIdx = currentIdx - 1;
  }

  set({
    openLetters: nextOpen,
    activeLetterIdx: nextIdx,
  });
},

// Routing Queue State
routingQueue: [],
addToRoutingQueue: (item: RoutingQueueItem) => {
  // Guard: check if the target letter tab still exists
  const letterExists = get().openLetters.some((l) => l.letterId ===
item.targetLetterId);
  if (!letterExists) {
    console.warn('Target letter tab is closed. Cannot route.');
```

```

    throw new Error('Letter tab is closed. Route to an active letter instead.');
```

```

    }

    set((state) => ({
      routingQueue: [...state.routingQueue, item],
    }));
  },
  removeFromRoutingQueue: (itemId: string) => {
    set((state) => ({
      routingQueue: state.routingQueue.filter((item) => item.id !== itemId),
    }));
  },

  // Crawler State
  crawlerFilters: {
    startDate: null,
    endDate: null,
    category: null,
    searchQuery: '',
  },
  setCrawlerFilters: (filters) => {
    set((state) => ({
      crawlerFilters: {
        ...state.crawlerFilters,
        ...filters,
      },
    }));
  },
}),
{
  name: 'journal-workspace-store',
  storage: createJSONStorage(() => sqliteStorage),
  // Partialize: only persist layout metadata, not in-memory queue or penpal list
  partialize: (state) => ({
    openLetters: state.openLetters,
    activeLetterIdx: state.activeLetterIdx,
    crawlerFilters: state.crawlerFilters,
  }),
}
)
);

```

--- END_FILE ---

--- START_FILE: src/styles/global.css ---

```

/*
 * Global Styles: Journal-to-Penpal Pipeline
 * Imports design tokens and sets base resets.
 */

```

```
@import "../tokens.css";
```

```

/* — Reset — */
*,
*::before,
*::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html {
  font-size: var(--text-base);
  line-height: var(--leading-base);
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

body {
  font-family: var(--font-family);
  font-size: var(--text-base);
  font-weight: var(--weight-regular);
  line-height: var(--leading-base);
  color: var(--color-text-primary);
  background-color: var(--color-background);
  overflow: hidden; /* Tauri manages window scrolling */
}

#root {
  width: 100vw;
  height: 100vh;
  overflow: hidden;
}

/* — Focus States (Accessibility – FRONTEND_GUIDELINES.md) — */
:focus-visible {
  outline: 2px solid var(--color-primary);
  outline-offset: 2px;
}

/* — App Shell Layout (Three-panel) — */
.app-shell {
  display: grid;
  grid-template-columns:
    minmax(var(--panel-left-min), var(--panel-left-default))
    minmax(var(--panel-center-min), 1fr)
    minmax(var(--panel-right-min), var(--panel-right-default));
  height: 100vh;
  overflow: hidden;
}

.panel-left {
  background: var(--color-surface);
  border-right: 1px solid var(--color-border);
}

```

```

    overflow-y: auto;
}

.panel-center {
    background: var(--color-surface);
    overflow: hidden;
    display: flex;
    flex-direction: column;
}

.panel-right {
    background: var(--color-surface);
    border-left: 1px solid var(--color-border);
    overflow-y: auto;
}

/* — Scrollbar Styling — */
::-webkit-scrollbar {
    width: 6px;
}

::-webkit-scrollbar-track {
    background: transparent;
}

::-webkit-scrollbar-thumb {
    background: var(--color-border);
    border-radius: var(--radius-full);
}

::-webkit-scrollbar-thumb:hover {
    background: var(--color-text-disabled);
}

/* — Selection — */
::selection {
    background: color-mix(in srgb, var(--color-primary) 30%, transparent);
    color: var(--color-text-primary);
}

/* — Typography Helpers — */
h1, h2, h3, h4, h5, h6 {
    font-weight: var(--weight-semibold);
    line-height: var(--leading-xl);
}

a {
    color: var(--color-primary);
    text-decoration: none;
}

a:hover {
    color: var(--color-primary-hover);
}

```

```
}
```

```
--- END_FILE ---
```

```
--- START_FILE: src/styles/tokens.css ---
```

```
/*
 * Design Tokens: Journal-to-Penpal Pipeline
 * Source: DESIGN_SYSTEM.md – Phase 6 Spec Formalization
 *
 * Dark theme is default. Light theme via [data-theme="light"] or prefers-color-scheme.
 */

:root {
  /* — Colors (Dark – default) — */
  --color-primary: #7C6AEF;
  --color-primary-hover: #9B8CF5;
  --color-surface: #1E1E2E;
  --color-surface-elevated: #2A2A3E;
  --color-background: #141420;
  --color-text-primary: #E4E4F0;
  --color-text-secondary: #9898B0;
  --color-text-disabled: #5A5A70;
  --color-border: #2E2E42;
  --color-error: #FF6B6B;
  --color-success: #51CF66;
  --color-warning: #FFD43B;
  --color-presence: #4ECDC4;
  --color-reminiscence: #FF8A65;
  --color-presence-tint: rgba(78, 205, 196, 0.08);
  --color-reminiscence-tint: rgba(255, 138, 101, 0.08);
  --color-scaffold-tint: rgba(124, 106, 239, 0.08);
  --color-routed-border: #7C6AEF;

  /* — Typography — */
  --font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  --text-xs: 11px;
  --text-sm: 13px;
  --text-base: 15px;
  --text-lg: 17px;
  --text-xl: 20px;
  --text-2xl: 24px;

  --weight-regular: 400;
  --weight-medium: 500;
  --weight-semibold: 600;
  --weight-bold: 700;

  --leading-xs: 1.4;
```

```

--leading-sm: 1.5;
--leading-base: 1.6;
--leading-lg: 1.5;
--leading-xl: 1.4;
--leading-2xl: 1.3;

/* — Spacing (4px base) — */
--space-1: 4px;
--space-2: 8px;
--space-3: 12px;
--space-4: 16px;
--space-6: 24px;
--space-8: 32px;
--space-12: 48px;

/* — Border Radius — */
--radius-sm: 4px;
--radius-md: 8px;
--radius-lg: 12px;
--radius-xl: 16px;
--radius-full: 9999px;

/* — Shadows (Dark) — */
--shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.3);
--shadow-md: 0 4px 12px rgba(0, 0, 0, 0.4);
--shadow-lg: 0 8px 24px rgba(0, 0, 0, 0.5);

/* — Animations — */
--transition-fast: 120ms ease-out;
--transition-base: 200ms ease-in-out;
--transition-slow: 350ms ease-in-out;

/* — Icon Sizes — */
--icon-sm: 14px;
--icon-md: 18px;
--icon-lg: 24px;

/* — Panel Widths — */
--panel-left-min: 280px;
--panel-left-default: 320px;
--panel-center-min: 400px;
--panel-right-min: 260px;
--panel-right-default: 300px;
}

/* — Light Theme — */
[data-theme="light"],
:root:has(meta[name="color-scheme"][content="light"]) {
  --color-primary: #6C5CE7;
  --color-primary-hover: #5A4BD5;
  --color-surface: #FFFFFF;
  --color-surface-elevated: #F8F9FA;
  --color-background: #F0F1F5;

```

```

--color-text-primary: #1A1A2E;
--color-text-secondary: #6B7280;
--color-text-disabled: #C0C4CC;
--color-border: #E2E4E8;
--color-error: #E53E3E;
--color-success: #38A169;
--color-warning: #D69E2E;
--color-presence: #319795;
--color-reminiscence: #DD6B20;
--color-presence-tint: rgba(49, 151, 149, 0.08);
--color-reminiscence-tint: rgba(221, 107, 32, 0.08);
--color-scaffold-tint: rgba(108, 92, 231, 0.06);
--color-routed-border: #6C5CE7;

--shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.08);
--shadow-md: 0 4px 12px rgba(0, 0, 0, 0.12);
--shadow-lg: 0 8px 24px rgba(0, 0, 0, 0.16);
}

@media (prefers-color-scheme: light) {
  :root:not([data-theme="dark"]) {
    --color-primary: #6C5CE7;
    --color-primary-hover: #5A4BD5;
    --color-surface: #FFFFFF;
    --color-surface-elevated: #F8F9FA;
    --color-background: #F0F1F5;
    --color-text-primary: #1A1A2E;
    --color-text-secondary: #6B7280;
    --color-text-disabled: #C0C4CC;
    --color-border: #E2E4E8;
    --color-error: #E53E3E;
    --color-success: #38A169;
    --color-warning: #D69E2E;
    --color-presence: #319795;
    --color-reminiscence: #DD6B20;
    --color-presence-tint: rgba(49, 151, 149, 0.08);
    --color-reminiscence-tint: rgba(221, 107, 32, 0.08);
    --color-scaffold-tint: rgba(108, 92, 231, 0.06);
    --color-routed-border: #6C5CE7;

    --shadow-sm: 0 1px 2px rgba(0, 0, 0, 0.08);
    --shadow-md: 0 4px 12px rgba(0, 0, 0, 0.12);
    --shadow-lg: 0 8px 24px rgba(0, 0, 0, 0.16);
  }
}

```

--- END_FILE ---

--- START_FILE: src/types/index.ts ---

```
export interface ThoughtUnit {
```

```
    id: string;
    date: string | null; // ISO date "YYYY-MM-DD"
    content: string;
    category: 'presence' | 'reminiscence' | 'uncategorized';
    sourceFilePath: string;
    sourceLineNumber: number;
    formatVersion: number;
    createdAt: string;
}

export interface JournalSource {
    id: string;
    filePath: string;
    fileName: string;
    lastModified: string;
    lastImported: string;
    formatVersion: number;
    entryCount: number;
}

export interface ImportResult {
    entryCount: number;
    formatVersion: number;
    warnings: string[];
}

export interface Penpal {
    id: string;
    name: string;
    country: string;
    interests: string;
    topics: string;
    notes: string;
    createdAt: string;
    updatedAt: string;
    letterCount?: number;
    lastLetterDate?: string | null;
}

export interface Correspondence {
    id: string;
    penpalId: string;
    direction: 'sent' | 'received';
    content: string;
    letterDate: string;
    importedAt: string;
}

export interface Letter {
    id: string;
    penpalId: string;
    status: 'draft' | 'sent';
    createdAt: string;
```



```

    updatedAt: string;
}

export interface WorkspaceState {
  openLetterIds: string; // JSON string of string[]
  activeLetterId: string | null;
  crawlerState: string; // JSON string of filters, etc.
}

export interface RoutingQueueItem {
  id: string;
  sourceThoughtUnit: ThoughtUnit;
  targetLetterId: string;
  insertPosition: 'end' | 'cursor';
  timestamp: number;
  blockType?: 'routedEntry' | 'scaffoldBlock';
  scaffoldId?: string;
  sectionLabel?: string;
}

--- END_FILE ---

--- START_FILE: src/utils/serialization.ts ---

/**
 * Recursively serializes a list of BlockNote blocks into clean plain text.
 * Strips all metadata, labels, and badges, and joins paragraphs/blocks with double
 * newlines.
 */
export function serializeToPlainText(blocks: any[]): string {
  if (!Array.isArray(blocks)) return '';

  const lines: string[] = [];

  const processBlock = (block: any) => {
    let text = '';
    if (block.type === 'routedEntry') {
      if (block.content && Array.isArray(block.content)) {
        text = block.content.map((inline: any) => inline.text || '').join('');
      } else {
        text = block.props?.entryContent || '';
      }
    } else if (block.type === 'scaffoldBlock') {
      const label = block.props?.sectionLabel || block.props?.title || 'Scaffold
Section';
      const labelText = label ? `[Scaffold: ${label}]` : '';
      let inlineText = '';
      if (block.content && Array.isArray(block.content)) {
        inlineText = block.content.map((inline: any) => inline.text || '').join('');
      }
      text = labelText + inlineText;
    }
  };

```

```

    } else if (block.content && Array.isArray(block.content)) {
      text = block.content.map((inline: any) => inline.text || '').join('');
    } else if (typeof block.content === 'string') {
      text = block.content;
    }
    if (text.trim() !== '') {
      lines.push(text);
    }
    if (block.children && Array.isArray(block.children)) {
      block.children.forEach(processBlock);
    }
  };

  blocks.forEach(processBlock);
  return lines.join('\n\n');
}

```

--- END_FILE ---

--- START_FILE: src/vite-env.d.ts ---

/// <reference types="vite/client" />

--- END_FILE ---

--- START_FILE: vite.config.ts ---

```

import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

// https://v2.tauri.app/start/frontend/vite/
export default defineConfig({
  plugins: [react()],

  // Prevent vite from obscuring rust errors
  clearScreen: false,

  server: {
    // Tauri expects a fixed port; fail if that port is not available
    port: 1420,
    strictPort: true,
    // Allow Tauri's dev server to connect
    host: "localhost",
  },

  // Env variables starting with TAURI_ are available in the frontend
  envPrefix: ["VITE_", "TAURI_"],

  build: {

```

```
// Tauri uses Chromium on Windows and WebKit on macOS/Linux
target: process.env.TAURI_ENV_PLATFORM === "windows" ? "chrome105" : "safari14",
// Don't minify for debug builds
minify: !process.env.TAURI_ENV_DEBUG ? "esbuild" : false,
// Produce sourcemaps for debug builds
sourcemap: !!process.env.TAURI_ENV_DEBUG,
},
});
```

--- END_FILE ---

--- START_FILE: vibe_snapshot_env.txt ---

VIBECODE SNAPSHOT ENVIRONMENT

Generated: 2026-05-30T02:55:38.653592

Platform: Windows 11

Python Version: 3.13.13

Executable: C:\Users\USER\Desktop\APPS\vibecode
(old)\vibecode-project\.venv\Scripts\python.exe

aiohappyeyeballs==2.6.2
aiohttp==3.13.5
aiosignal==1.4.0
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.13.0
attrs==26.1.0
bcrypt==5.0.0
brotli==1.2.0
build==1.5.0
certifi==2026.5.20
cffi==2.0.0
charset-normalizer==3.4.7
chromadb==1.5.9
click==8.4.1
colorama==0.4.6
cryptography==48.0.0
cssselect2==0.9.0
defusedxml==0.7.1
distro==1.9.0
durationpy==0.10
filelock==3.29.0
flatbuffers==25.12.19
fonttools==4.63.0
fpdf2==2.8.7
frozenset==1.8.0
fsspec==2026.4.0
google-auth==2.53.0
google-genai==2.6.0
googleapis-common-protos==1.75.0
grpcio==1.80.0

h11==0.16.0
hf-xet==1.5.0
httpcore==1.0.9
httptools==0.7.1
httpx==0.28.1
httpx-sse==0.4.3
huggingface_hub==1.16.1
idna==3.16
importlib_resources==7.1.0
jaraco.classes==3.4.0
jaraco.context==6.1.2
jaraco.functools==4.5.0
jiter==0.15.0
jsonschema==4.26.0
jsonschema-specifications==2025.9.1
keyring==25.7.0
kubernetes==36.0.0
Markdown==3.10.2
markdown-it-py==4.2.0
mcp==1.27.1
mdurl==0.1.2
mistune==3.2.1
mmh3==5.2.1
more-itertools==11.1.0
multidict==6.7.1
numpy==2.4.6
oauthlib==3.3.1
onnxruntime==1.26.0
openai==2.38.0
opentelemetry-api==1.42.1
opentelemetry-exporter-otlp-proto-common==1.42.1
opentelemetry-exporter-otlp-proto-grpc==1.42.1
opentelemetry-proto==1.42.1
opentelemetry-sdk==1.42.1
opentelemetry-semantic-conventions==0.63b1
orjson==3.11.9
overrides==7.7.0
packaging==26.2
pathspec==1.1.1
pillow==12.2.0
propcache==0.5.2
protobuf==6.33.6
pyasn1==0.6.3
pyasn1_modules==0.4.2
pybase64==1.4.3
pycparser==3.0
pydantic==2.13.4
pydantic-settings==2.14.1
pydantic_core==2.46.4
pydyf==0.12.1
Pygments==2.20.0
PyJWT==2.13.0
pypdf==6.12.1

```
pyphen==0.17.2
PyPika==0.51.1
pyproject_hooks==1.2.0
PyQt6==6.11.0
PyQt6-Qt6==6.11.1
PyQt6_sip==13.11.1
python-dateutil==2.9.0.post0
python-dotenv==1.2.2
python-multipart==0.0.29
pywin32==311
pywin32-ctypes==0.2.3
PyYAML==6.0.3
referencing==0.37.0
requests==2.34.2
requests-oauthlib==2.0.0
rich==15.0.0
rpds-py==0.30.0
shellingham==1.5.4
six==1.17.0
sniffio==1.3.1
sse-starlette==3.4.4
starlette==1.1.0
tenacity==9.1.4
tinycss2==1.5.1
tinyhtml5==2.1.0
tokenizers==0.23.1
tqdm==4.67.3
typer==0.25.1
typing-inspection==0.4.2
typing_extensions==4.15.0
urllib3==2.7.0
uvicorn==0.48.0
-e
git+https://github.com/ajpop3y/vibecode-project.git@92c71967ae074c7e10de604d3ba13ffd41d4
36be#egg=vibecode
watchfiles==1.2.0
weasyprint==68.1
webencodings==0.5.1
websocket-client==1.9.0
websockets==16.0
yarl==1.24.2
zopfli==0.4.1
```

```
--- END_FILE ---
```

